



## 자바응용SW(앱)개발자양성

### Chapter 05

# 레이아웃

백제직업전문학교

김영준 강사



# 대표적인 레이아웃

| 레이아웃 이름                       | 설 명                                                                                                                             |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 제약 레이아웃<br>(ConstraintLayout) | 제약 조건(Constraint) 기반 모델<br>제약 조건을 사용해 화면을 구성하는 방법<br>안드로이드 스튜디오에서 자동으로 설정하는 디폴트 레이아웃                                            |
| 리니어 레이아웃<br>(LinearLayout)    | 박스(Box) 모델<br>한 쪽 방향으로 차례대로 뷰를 추가하며 화면을 구성하는 방법<br>뷰가 차지할 수 있는 사각형 영역을 할당                                                       |
| 상대 레이아웃<br>(RelativeLayout)   | 규칙(Rule) 기반 모델<br>부모 컨테이너나 다른 뷰와의 상대적 위치로 화면을 구성하는 방법                                                                           |
| 프레임 레이아웃<br>(FrameLayout)     | 싱글(Single) 모델<br>가장 상위에 있는 하나의 뷰 또는 뷰그룹만 보여주는 방법<br>여러 개의 뷰가 들어가면 중첩하여 쌓게 됨. 가장 단순하지만 여러 개의 뷰를 중첩한 후 각 뷰를 전환하여 보여주는 방식으로 자주 사용함 |
| 테이블 레이아웃<br>(TableLayout)     | 격자(Grid) 모델<br>격자 모양의 배열을 사용하여 화면을 구성하는 방법<br>HTML에서 많이 사용하는 정렬 방식과 유사하지만 많이 사용하지는 않음                                           |

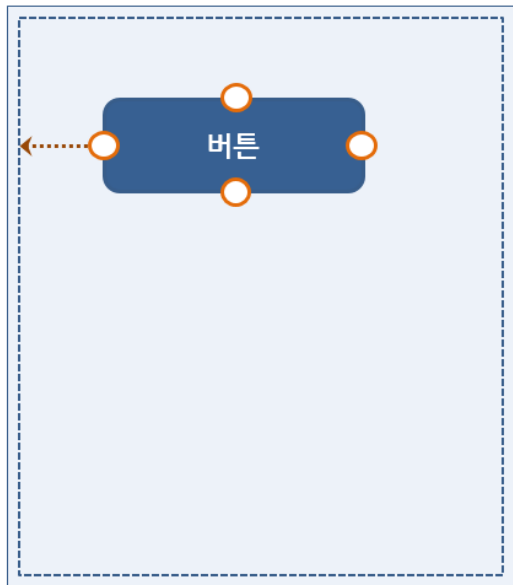


# 연결선으로 만드는 제약조건

- 뷰의 위, 아래, 왼쪽, 오른쪽의 연결점을 부모 레이아웃의 벽면과 연결하면 제약조건 생성
- 같은 레이아웃 안에 들어있는 다른 뷰와 연결 가능

제약조건(Constraint)

버튼의 왼쪽을 부모 레이아웃과 연결해 주세요.



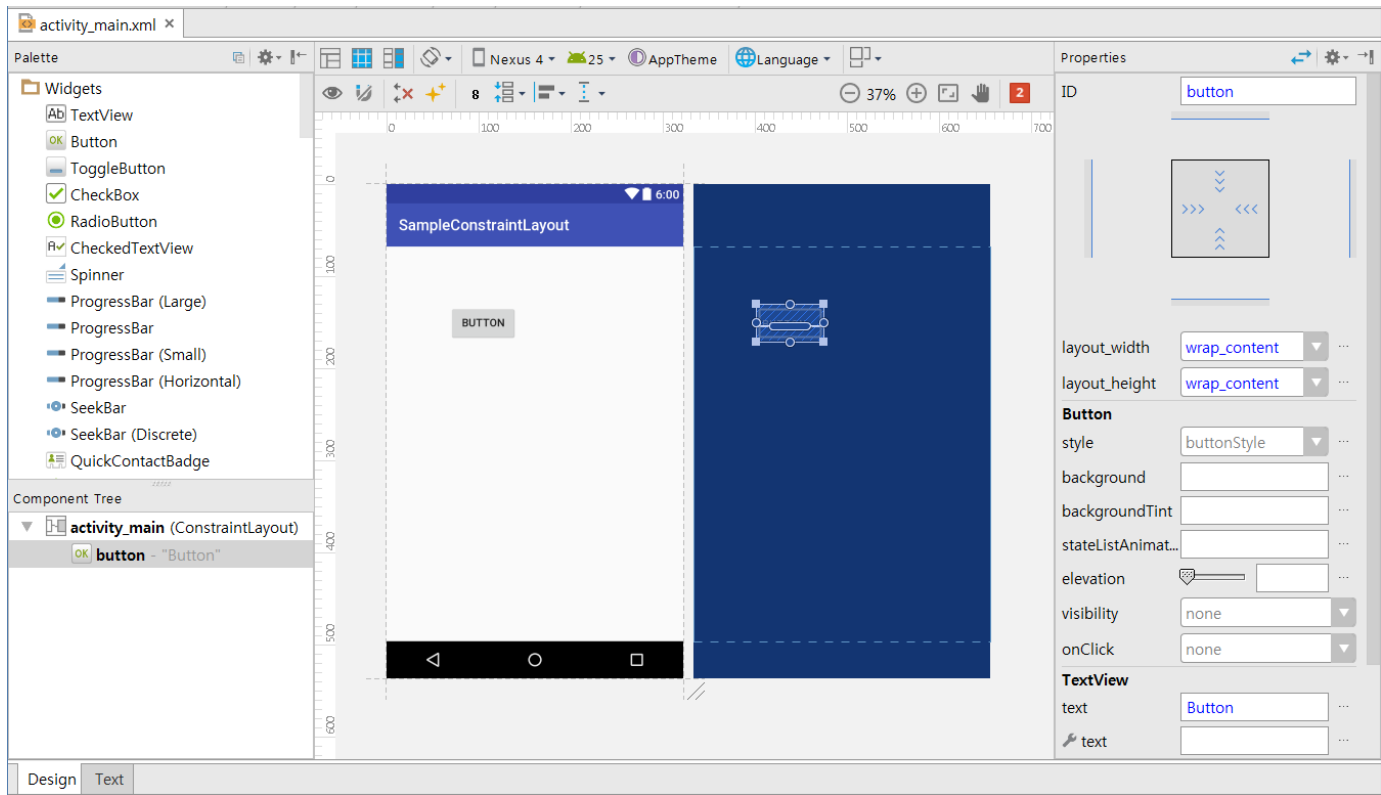
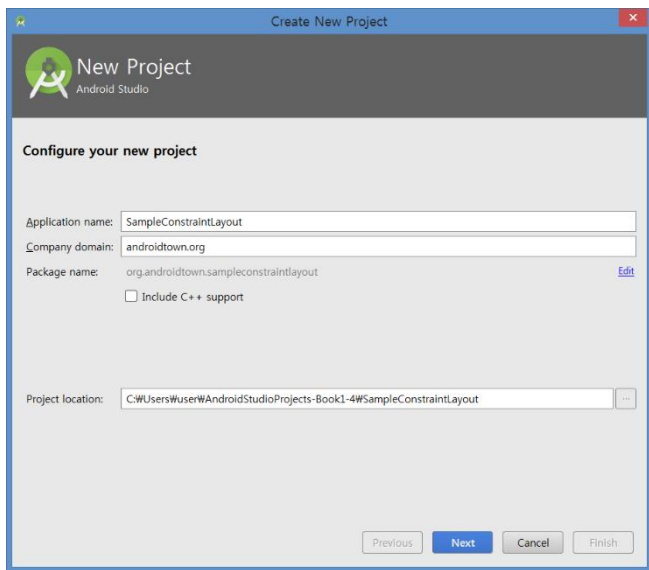
## • 연결점을 연결할 수 있는 타깃

- 같은 레이아웃 안에 들어 있는 다른 뷰의 연결점
- 부모 레이아웃의 연결점
- 가이드라인(Guideline)



# 새로운 프로젝트 생성

- SampleConstraintLayout이라는 이름으로 새로운 프로젝트 생성
- 화면 왼쪽 윗부분에 버튼 추가





# 연결선 생성

- 왼쪽과 위쪽에 있는 연결점을 부모 레이아웃의 벽면과 연결

The image displays three sequential screenshots from the Android Studio IDE, illustrating the process of creating constraints for a button in a layout.

**Left Screenshot:** Shows a layout editor with a button. A yellow callout box says "Click To Delete Left Constraint". The Properties panel on the right shows the button's properties, including layout\_width, layout\_height, style, background, backgroundTint, stateListAnimator, elevation, visibility, onClick, and TextView text.

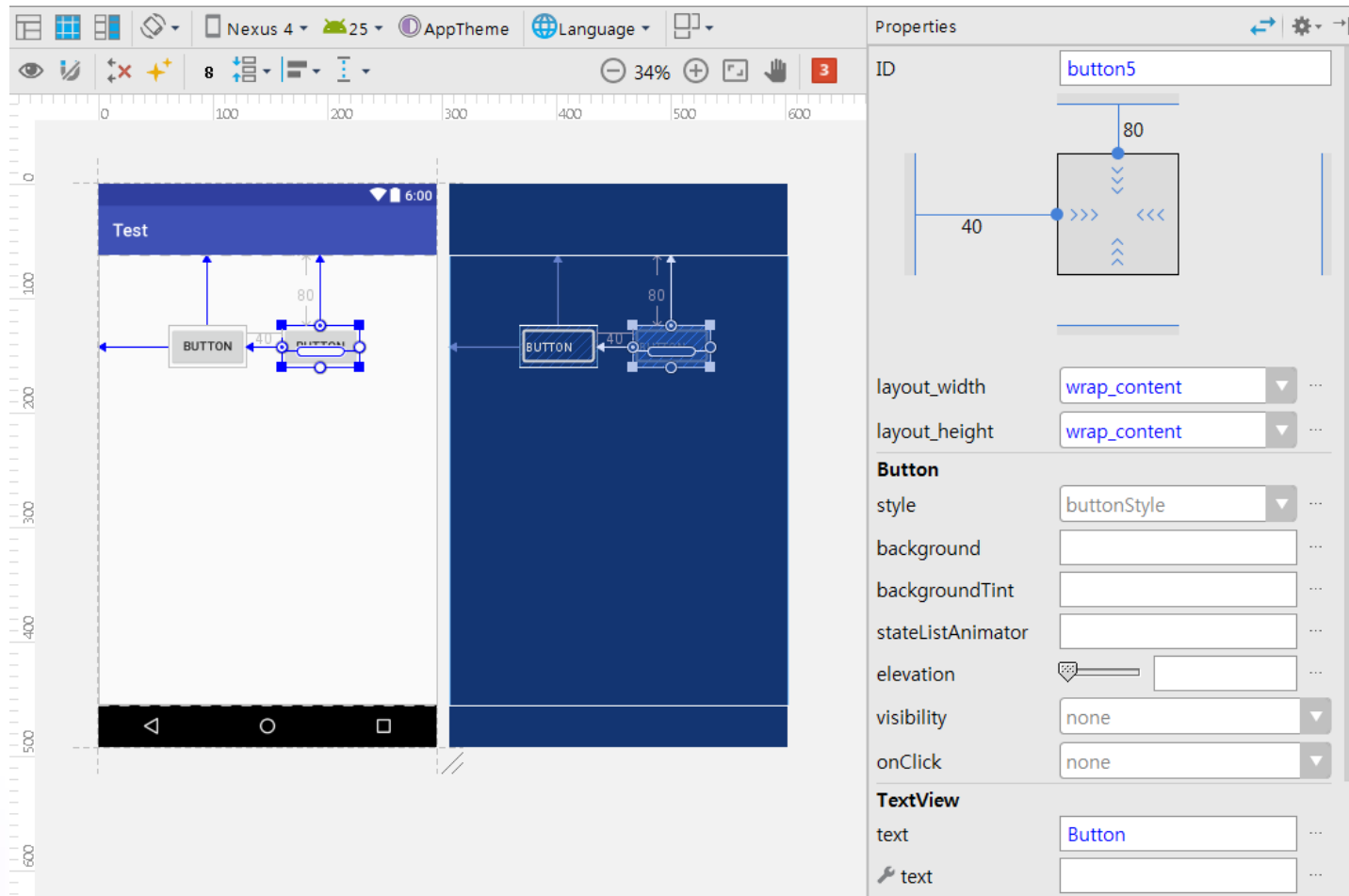
**Middle Screenshot:** Shows the same layout editor. The button is now constrained to the left and top walls of the parent layout. Dimension lines indicate the constraints are 80 units from the walls.

**Right Screenshot:** Shows the Properties panel for the button. The layout\_width and layout\_height are set to "wrap\_content". The style is "buttonStyle". The text is "Button".



# 버튼 하나 더 추가하고 연결선 생성

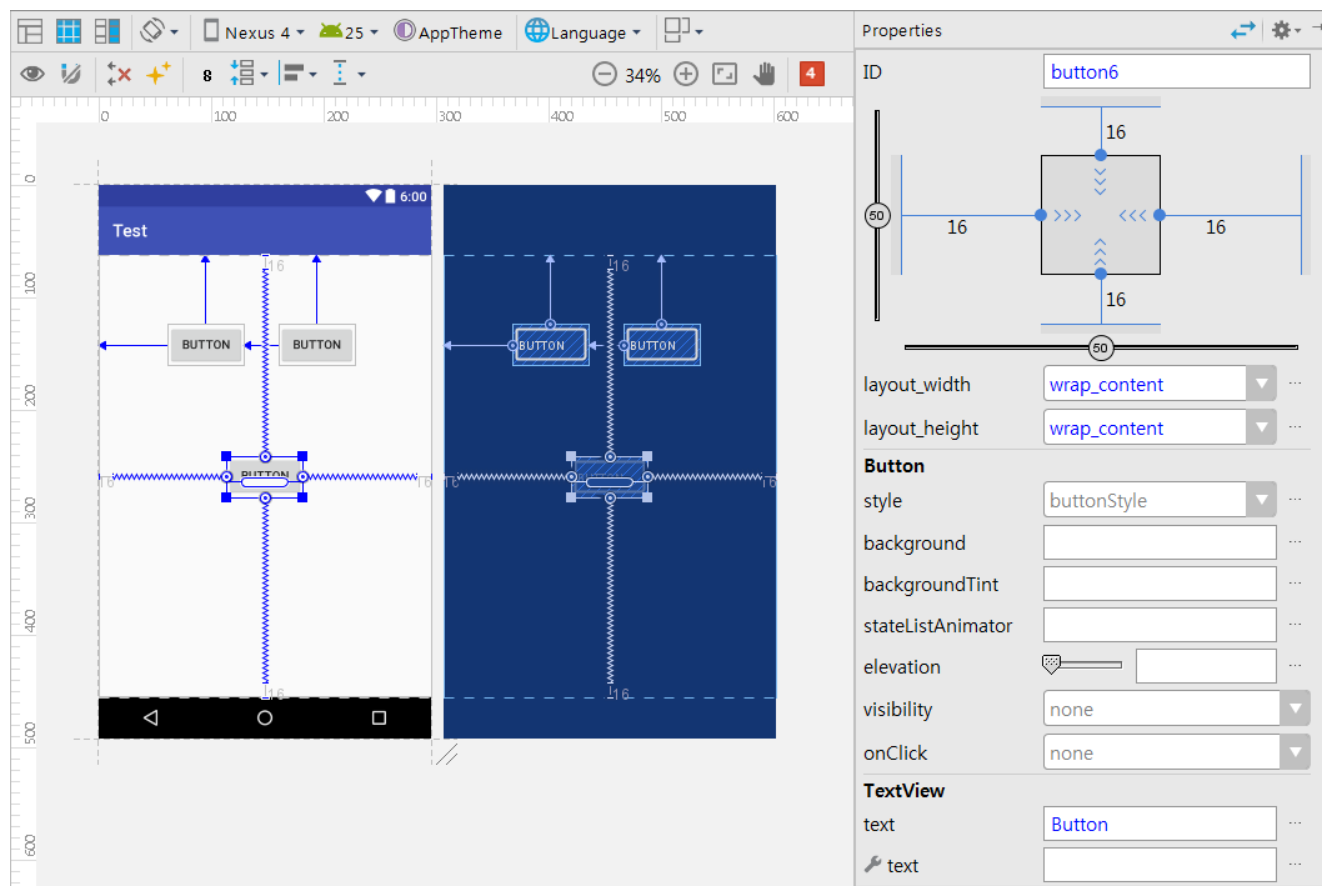
- 버튼 추가 후 부모 레이아웃 및 기존 버튼과 연결선 생성





# 화면 가운데에 뷰 배치하기

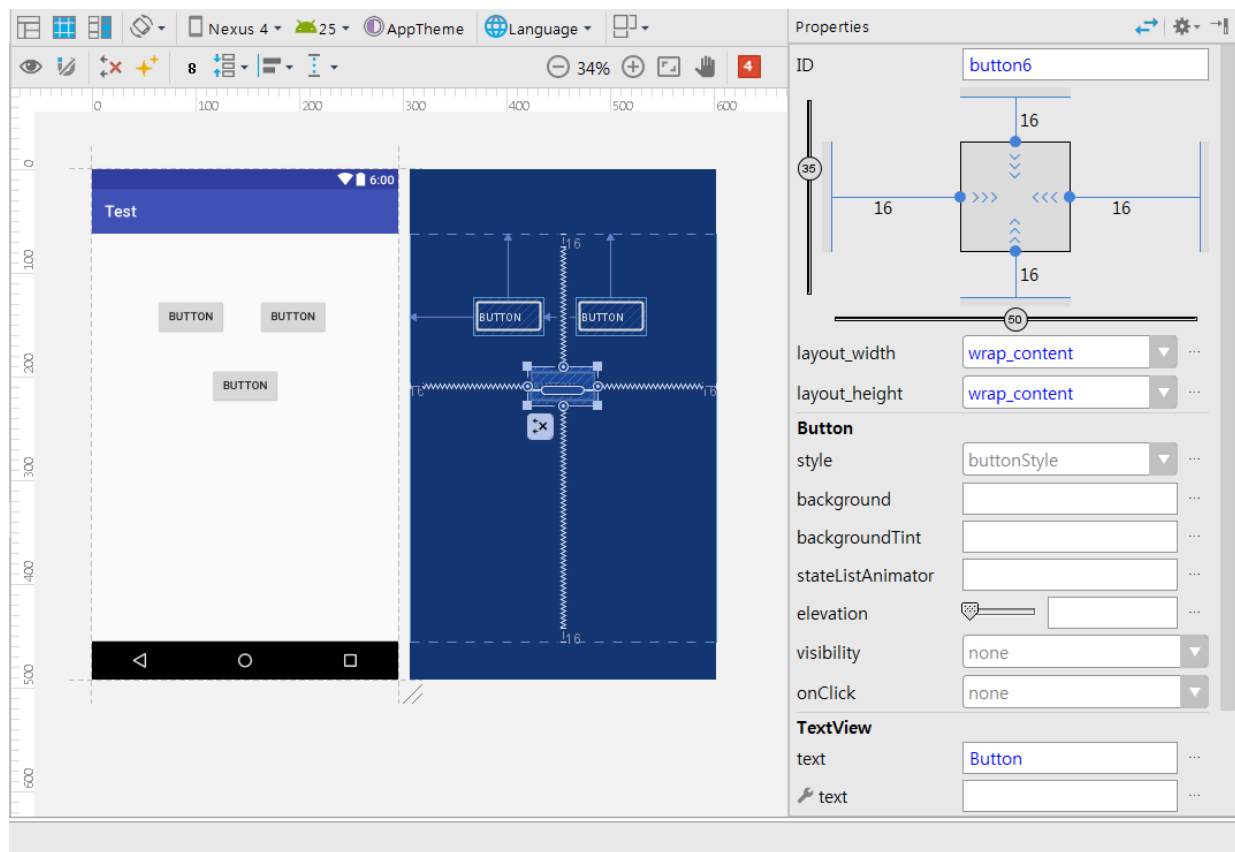
- 버튼 추가 후 위, 아래, 왼쪽, 오른쪽 모두 부모 레이아웃과 연결선 생성
- 좌우 또는 위,아래를 쌍으로 연결하면 그 가운데에 배치됨





# 바이어스 사용하기

- 양쪽 또는 상하로 연결된 경우 오른쪽 제약조건 미리보기 창에서 바이어스(Bias)를 이용해 위치 조정 가능

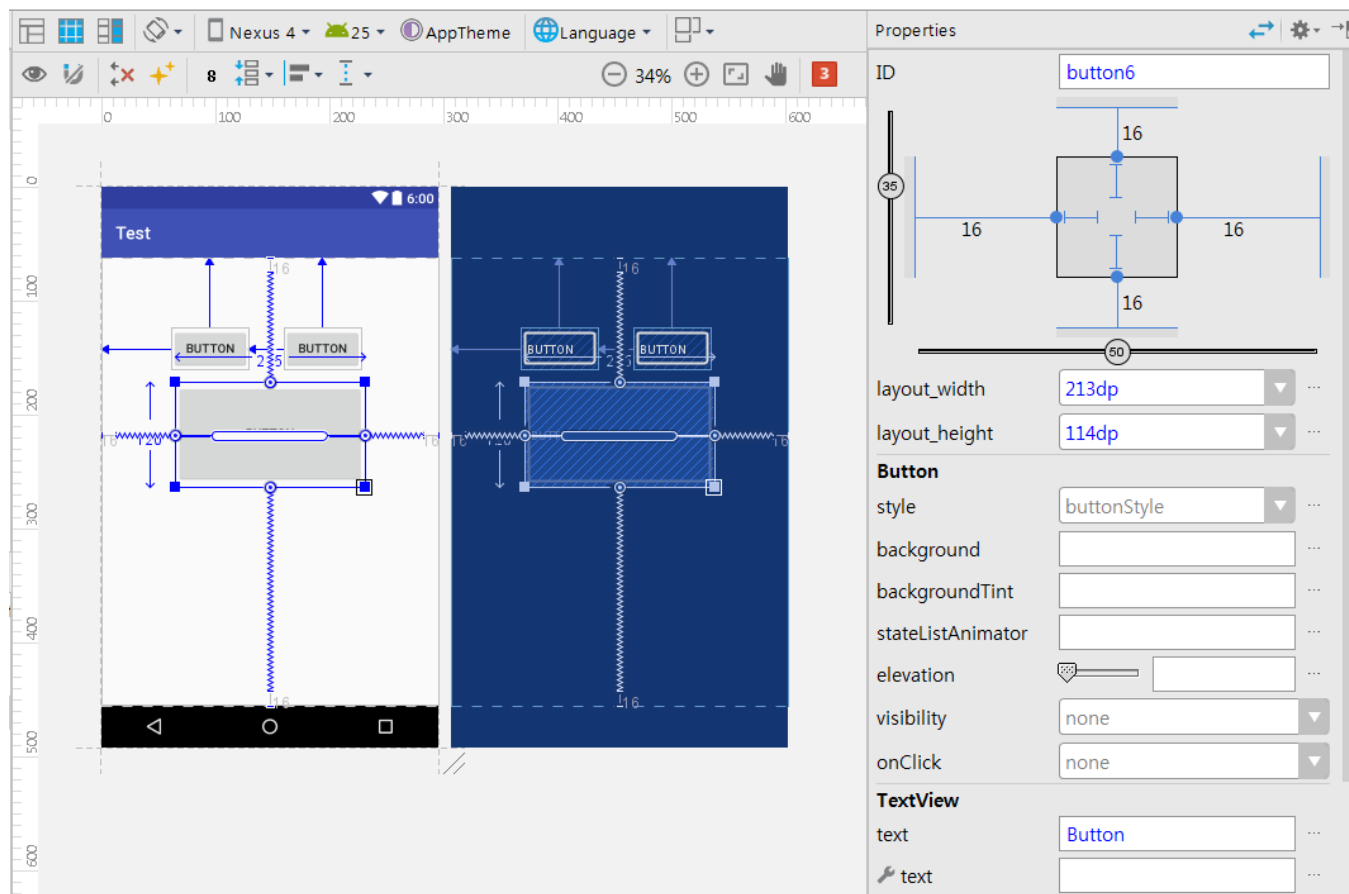






# 뷰의 크기 조정

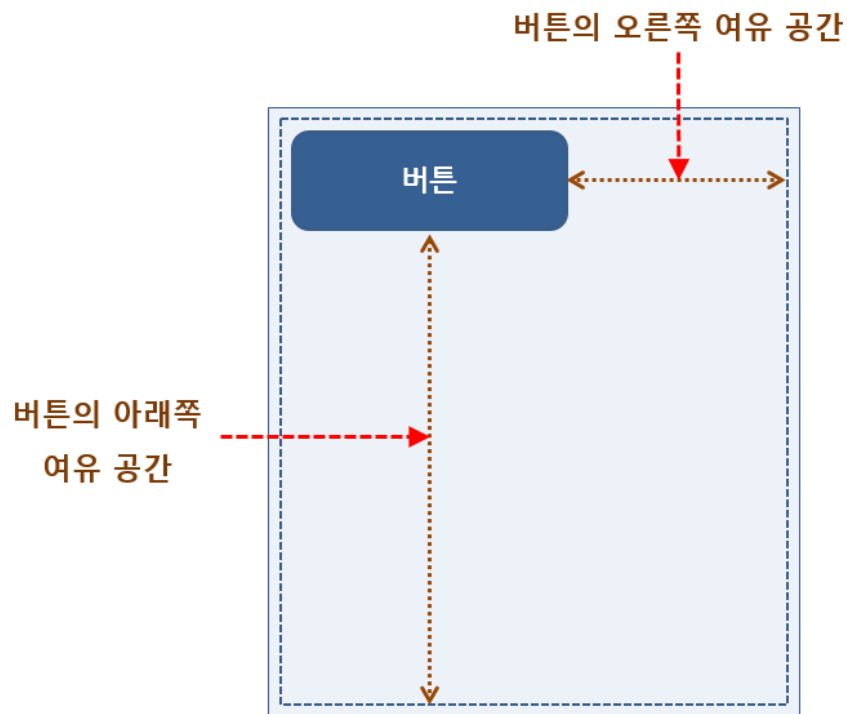
- 모서리에 있는 핸들(Handle)을 이용해 뷰의 크기 조정 가능





# 뷰가 차지할 수 있는 여유공간

- 부모 레이아웃 안에서 뷰가 차지할 수 있는 여유공간은 레이아웃에 의해 결정됨

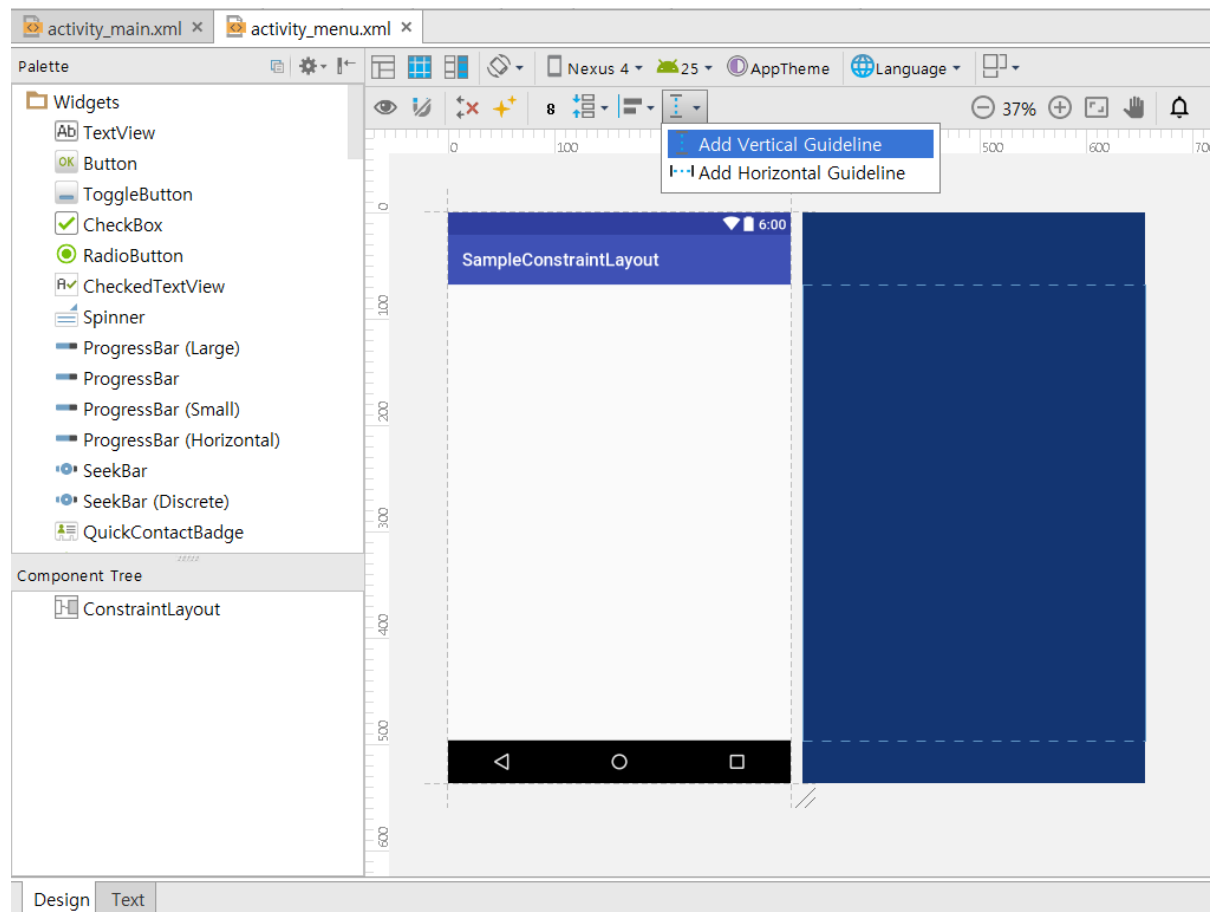
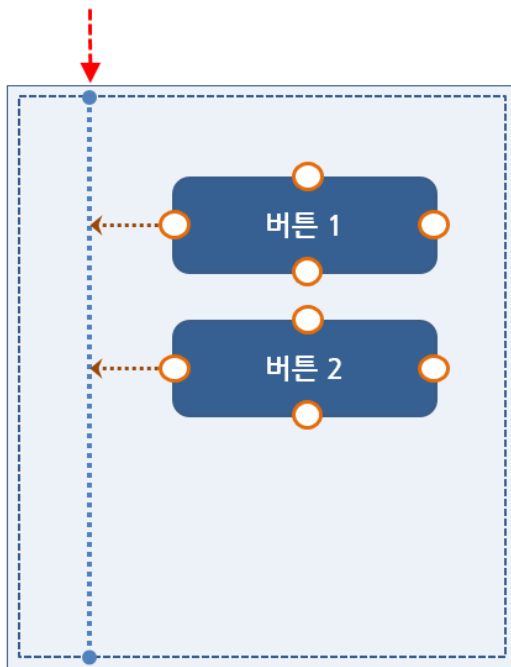




# 가이드라인 사용하기

- 일정 기준선으로 뷰를 정렬할 때나 기준선에 맞추어 추가할 때 사용됨

세로방향 가이드라인





# XML 코드로 만들어지는 가이드라인

- XML 레이아웃 파일에서 원본 XML을 보면 Guideline 태그로 만들어짐

```
<android.support.constraint.Guideline
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/guideline"
    android:orientation="vertical"
    tools:layout_editor_absoluteY="0dp"
    tools:layout_editor_absoluteX="20dp"
    app:layout_constraintGuide_begin="100dp" />
```

...



# 가이드라인에 맞추어 버튼 추가하기

- 가이드라인에 맞추어 버튼 세 개 추가

The screenshot shows the Android Studio IDE with the design tool active. The main canvas displays a 'SampleConstraintLayout' with three buttons stacked vertically. A dashed guideline is visible, and a new button is being added to the stack. The 'Properties' panel on the right shows the configuration for 'button4'.

**Properties Panel:**

- ID: button4
- layout\_width: wrap\_content
- layout\_height: wrap\_content
- Button**
  - style: buttonStyle
  - background:
  - backgroundTint:
  - stateListAnimat...:
  - elevation:
  - visibility: none
  - onClick: none
- TextView**
  - text: Button
  - text:



# 가이드라인의 원본 코드 살펴보기

- xmlns 로 시작하는 속성, android로 시작하는 속성 등이 있음

```
<?xml version="1.0" encoding="utf-8"?>  
<android.support.constraint.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">
```

...



# XML 코드의 접두어 의미

- XML 레이아웃 코드에서 접두어가 가지는 의미가 있음

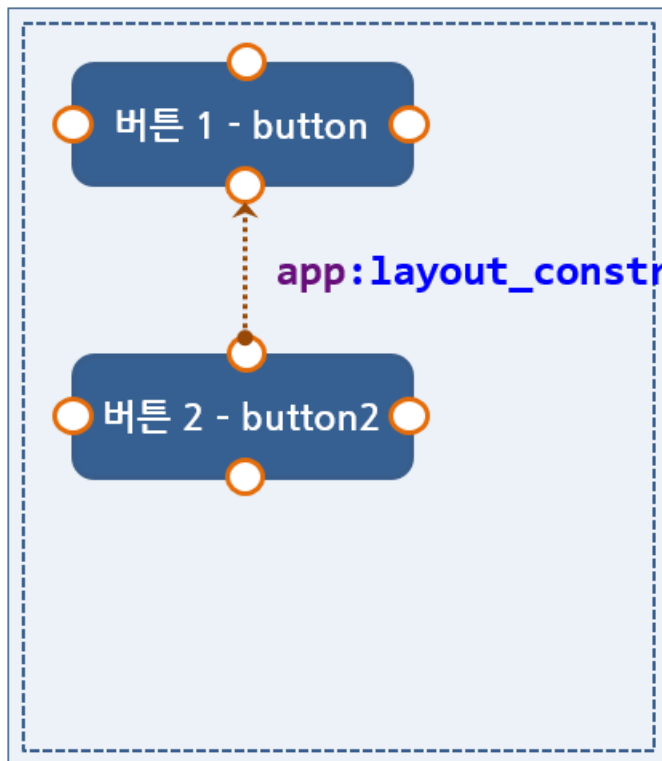
- (1) `xmlns:android` 안드로이드 기본 SDK에 포함되어 있는 속성을 사용합니다.
- (2) `xmlns:app` 프로젝트에서 사용하는 외부 라이브러리에 포함되어 있는 속성을 사용합니다.
- (3) `xmlns:tools` 안드로이드 스튜디오의 디자이너 도구 등에서 화면에 보여줄 때 사용합니다.  
이 속성은 앱이 실행될 때는 적용되지 않고 안드로이드 스튜디오에서만 적용됩니다.



# 연결선이 만들어내는 XML 속성의 형식

- 일정한 규칙으로 만들어짐

`layout_constraint[소스 뷰의 연결점]_[타겟 뷰의 연결점]="[타겟 뷰의 id]"`



`app:layout_constraintTop_toBottomOf="@+id/button"`





# 연결선이 만들어내는 XML 속성의 형식



```
<Button
    android:id="@+id/btn1"
    android:text="A"/>
<Button
    android:text="B"
    app:layout_constraintLeft_toRightOf="@+id/btn1"
    android:layout_marginLeft="20dp"/>
```



```
<Button
    android:id="@+id/btn1"
    android:text="A"
    android:visibility="gone"/>
<Button
    android:text="B"
    app:layout_constraintLeft_toRightOf="@+id/btn1"
    android:layout_marginLeft="20dp"/>
```



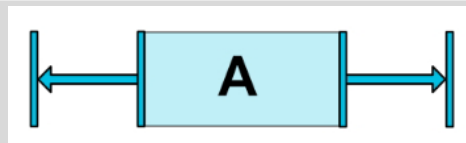
```
<Button
    android:id="@+id/btn1"
    android:text="A"
    android:visibility="gone"/>
<Button
    android:text="B"
    app:layout_constraintLeft_toRightOf="@+id/btn1"
    android:layout_marginLeft="20dp"
    app:layout_goneMarginLeft="0dp"/>
```



# 연결선이 만들어내는 XML 속성의 형식

**<Button**

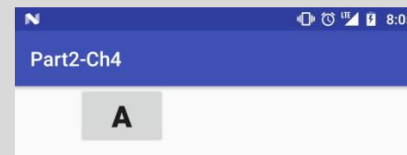
```
...  
app:layout_constraintLeft_toLeftOf="parent"  
app:layout_constraintRight_toRightOf="parent"/>
```



- layout\_constraintHorizontal\_bias: 가로 치우침 조절
- layout\_constraintVertical\_bias: 세로 치우침 조절

**<Button**

```
...  
app:layout_constraintLeft_toLeftOf="parent"  
app:layout_constraintRight_toRightOf="parent"  
app:layout_constraintHorizontal_bias="0.2"/>
```



값을 0.2로 지정하면 20%의 의미로 왼쪽에서 20% 위치



# 1. 레이아웃 개요 ▶ 레이아웃 기본 개념

## ❖ 레이아웃에서 자주 사용되는 속성

- ✓ orientation : 레이아웃 안에 배치할 위젯의 수직 또는 수평 방향을 설정
- ✓ gravity : 레이아웃 안에 배치할 위젯의 정렬 방향을 좌측, 우측, 중앙으로 설정
- ✓ padding : 레이아웃 안에 배치할 위젯의 여백을 설정
- ✓ layout\_weight : 레이아웃이 전체 화면에서 차지하는 공간의 가중 값을 설정  
여러 개의 레이아웃이 중복될 때 주로 사용
- ✓ baselineAligned : 레이아웃 안에 배치할 위젯들을 보기 좋게 정렬



# 1. 레이아웃 개요 ▶ 레이아웃의 종류

## ❖ 레이아웃의 종류



그림 5-1 레이아웃의 종류

## 2. 리니어레이아웃 ▶ 기본 리니어레이아웃 형태

### ❖ orientation 속성과 예제

- vertical은 수직 배열, horizontal은 수평배열

[예제 5-1] orientation 속성이 vertical 값인 XML 코드

```
1 <LinearLayout
2   android:orientation="vertical" >
3   <Button
4     android:layout_width="wrap_content"
5     android:layout_height="wrap_content"
6     android:text="Button" />
7   <TextView
8     android:text="TextView" />
9   <CheckBox
10    android:text="CheckBox" />
11  <RadioButton
12    android:text="RadioButton" />
13  <Switch
14    android:text="Switch" />
15 </LinearLayout>
```

Button  
TextView  
☐ CheckBox  
☐ RadioButton  
Switch OFF

[예제 5-2] orientation 속성이 horizontal 값인 XML 코드

```
1 <LinearLayout
2   android:orientation="horizontal" >
3   <Button
4     android:layout_width="wrap_content"
5     android:layout_height="wrap_content"
6     android:text="Button" />
7   <TextView
8     android:text="TextView" />
9   ~~~ 중간 생략 ~~~
10 </LinearLayout>
```

Button TextView ☐ CheckBox ☐ RadioButton

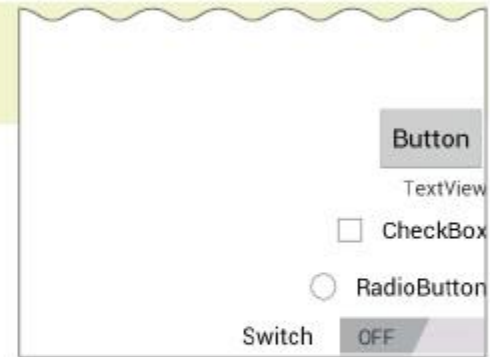
## 2. 리니어레이아웃 ▶ 기본 리니어레이아웃 형태

### ❖ gravity 속성

- ✓ gravity 속성은 레이아웃 안의 위젯들을 어디에 배치할 것인지를 결정

예제 5-3 gravity 속성 XML 코드

```
1 <LinearLayout
2     android:orientation="vertical"
3     android:gravity="right|bottom" >
4     <Button
5         android:layout_width="wrap_content"
6         android:layout_height="wrap_content"
7         android:text="Button" />
8     <TextView
9         android:text="TextView" />
10     ~~~ 중간 생략 ~~~
11 </LinearLayout>
```





## 2. 리니어레이아웃 ▶ 기본 리니어레이아웃 형태

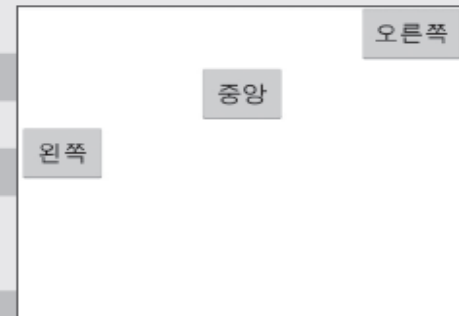
### ❖ layout\_gravity 속성

- layout\_gravity는 자신의 위치를 부모(주로 레이아웃)의 어디쯤에 위치할 것인지를 결정

### ❖ 예제

[예제 5-4] layout\_gravity 속성 XML 코드

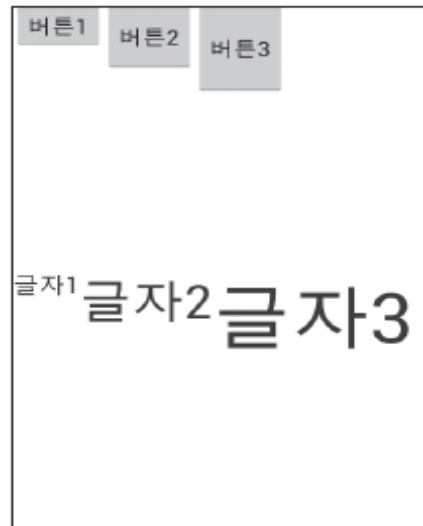
```
1 <LinearLayout
2   android:orientation="vertical" >
3   <Button
4     android:layout_gravity="right"
5     android:text="오른쪽" />
6   <Button
7     android:layout_gravity="center"
8     android:text="중앙" />
9   <Button
10    android:layout_gravity="left"
11    android:text="왼쪽" />
12 </LinearLayout>
```



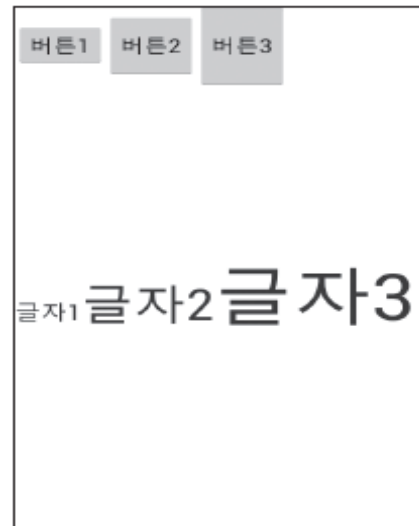
## 2. 리니어레이아웃 ▶ 기본 리니어레이아웃 형태

### ❖ baselineAligned 속성

- ✓ baselineAligned 속성은 크기가 다른 위젯들을 보기 좋게 정렬함
- ✓ true와 false 값을 가질 수 있음



(a) false로 지정



(b) true로 지정하거나 생략

[그림 5-3] baselineAligned 속성



## 2. 리니어레이아웃 ▶ 기본 리니어레이아웃 형태

### ❖ baselineAligned와 baselineAlignedChildIndex

```
res/layout/linearlayout_baseline_aligned_child_index.xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="160dp"
    android:orientation="horizontal"
    android:baselineAligned="true">

    <Button android:layout_width="wrap_content"
        android:layout_height="40dp"
        android:text="View1"/>

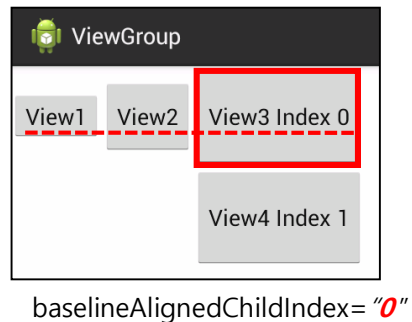
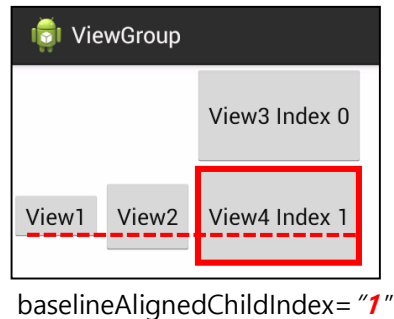
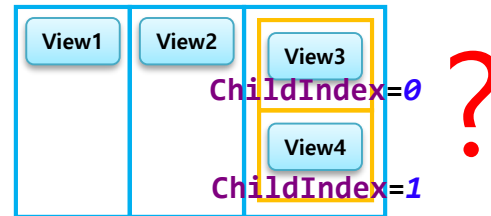
    <Button android:layout_width="wrap_content"
        android:layout_height="60dp"
        android:text="View2" />

    <LinearLayout android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:baselineAlignedChildIndex="1">

        <Button android:layout_width="wrap_content"
            android:layout_height="80dp"
            android:text="View3 Index 0" />

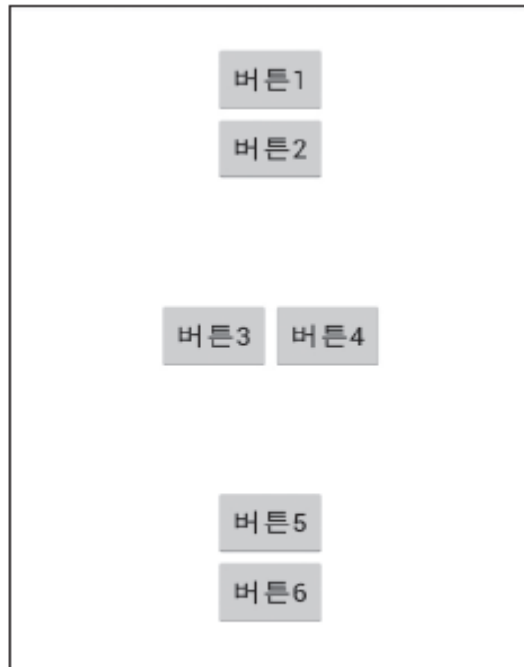
        <Button android:layout_width="wrap_content"
            android:layout_height="80dp"
            android:text="View4 Index 1" />
    </LinearLayout>
</LinearLayout>
```

#### ■ 어디에 맞춰야 할까?



## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

### ❖ 중복 리니어레이아웃 기본 형태



[그림 5-4] 다양한 배치의 레이아웃

```
<LinearLayout>

    <LinearLayout>
        위젯들...
    </LinearLayout>

    <LinearLayout>
        위젯들...
    </LinearLayout>

    <LinearLayout>
        위젯들...
    </LinearLayout>

</LinearLayout>
```

## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

### ❖ layout\_weight 속성

- 리니어레이아웃을 여러 개 사용하면, 각 레이아웃의 크기를 지정

### ❖ 예제 1 - 오류

[예제 5-5] 세 개의 레이아웃으로 구분한 XML 코드

```
1 <LinearLayout
2     android:layout_width="match_parent"
3     android:layout_height="match_parent"
4     android:orientation="vertical" >
5     <LinearLayout
6         android:layout_width="match_parent"
7         android:layout_height="match_parent"
8         android:gravity="center"
9         android:orientation="vertical" >
10         <Button
11             android:text="버튼1" />
12         <Button
13             android:text="버튼2" />
14     </LinearLayout>
```

```
15     <LinearLayout
16         android:layout_width="match_parent"
17         android:layout_height="match_parent"
18         android:background="#00FF00"
19         android:gravity="center"
20         android:orientation="horizontal" >
21         <Button
22             android:text="버튼3" />
23         <Button
24             android:text="버튼4" />
25     </LinearLayout>
26     <LinearLayout
27         android:layout_width="match_parent"
28         android:layout_height="match_parent"
29         android:background="#0000FF"
30         android:gravity="center"
31         android:orientation="vertical" >
32         <Button
33             android:text="버튼5" />
34         <Button
35             android:text="버튼6" />
36     </LinearLayout>
37 </LinearLayout>
```



## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

### ❖ 예제 2(오류)와 예제 3(정상)

[예제 5-6] layout\_height를 wrap\_content로 변경

```
1 <LinearLayout
2     android:orientation="vertical" >
3     <LinearLayout
4         android:layout_width="match_parent"
5         android:layout_height="wrap_content"
6         ~~~~ 중간 생략 ~~~~
7     </LinearLayout>
8     <LinearLayout
9         android:layout_width="match_parent"
10        android:layout_height="wrap_content"
11        ~~~~ 중간 생략 ~~~~
12    </LinearLayout>
13    <LinearLayout
14        android:layout_width="match_parent"
15        android:layout_height="wrap_content"
16        ~~~~ 중간 생략 ~~~~
17    </LinearLayout>
18 </LinearLayout>
```

[예제 5-7] layout\_weight를 1로 지정

```
1 <LinearLayout
2     android:orientation="vertical" >
3     <LinearLayout
4         android:layout_width="match_parent"
5         android:layout_height="match_parent"
6         android:layout_weight="1"
7         ~~~~ 중간 생략 ~~~~
8     </LinearLayout>
9     <LinearLayout
10        android:layout_width="match_parent"
11        android:layout_height="match_parent"
12        android:layout_weight="1"
13        ~~~~ 중간 생략 ~~~~
14    </LinearLayout>
15    <LinearLayout
16        android:layout_width="match_parent"
17        android:layout_height="match_parent"
18        android:layout_weight="1"
19        ~~~~ 중간 생략 ~~~~
20    </LinearLayout>
21 </LinearLayout>
```





## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

### ❖ layout\_weight 속성

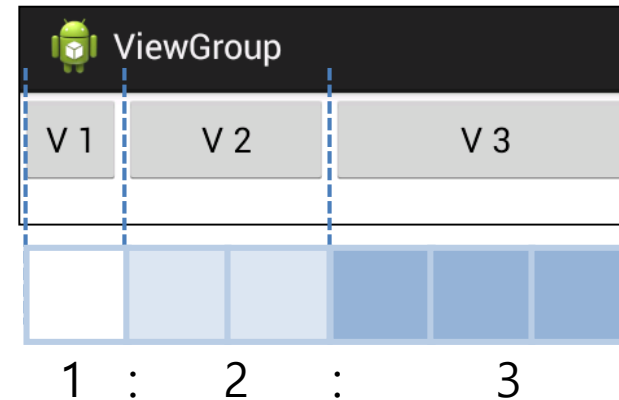
```
res/layout/linearlayout_layout_weight_1.xml
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="130dp"
    android:orientation="horizontal">

    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="V 1"/>

    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:text="V 2" />

    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="3"
        android:text="V 3" />

</LinearLayout>
```



## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

❗ 레이아웃의 유연성 속성은 기억해주세요.

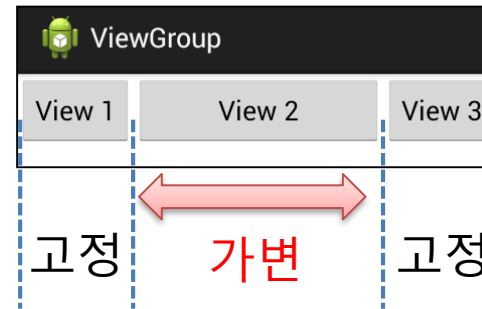
```
res/layout/linearlayout_layout_weight_2.xml
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="130dp"
    android:orientation="horizontal">

    <Button android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="View 1"/>

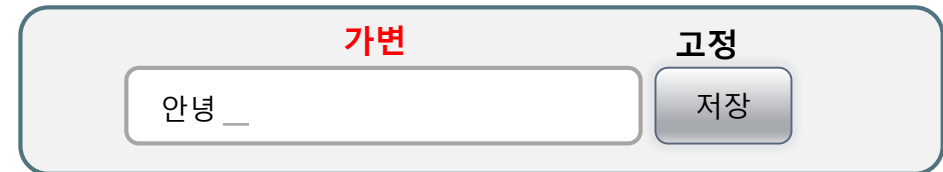
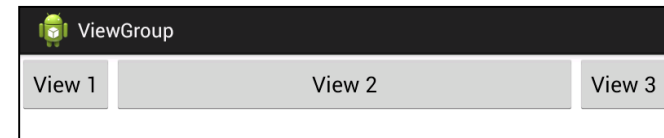
    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="View 2" />

    <Button android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="View 3" />

</LinearLayout>
```



단말기를 가로로 회전했을 때



LinearLayout LayoutParams의 layout\_weight 속성은 특정 자식뷰의 크기를 가변적으로 조절할 수 있기 때문에 다양한 화면 크기의 단말에서 유연하게 레이아웃을 유지할 수 있다.

매우 빈번히 사용되니 꼭 기억해두자.



## 2. 리니어레이아웃 ▶ 중복 리니어레이아웃 형태

LinearLayout LayoutParams 속성 –  
layout\_weight 속성과 LinearLayout의 weightSum 속성

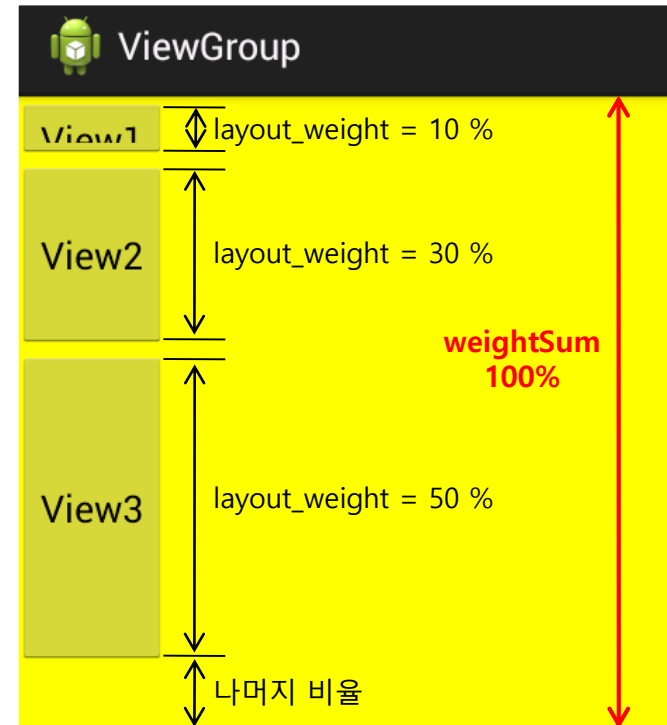
```
res/layout/linearlayout_measure_with_largest_child.xml
<LinearLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="300dp"
    android:background="#FF0"
    android:orientation="vertical"
    android:weightSum="100">

    <Button
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_weight="10"
        android:text="View1"/>

    <Button
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_weight="30"
        android:text="View2" />

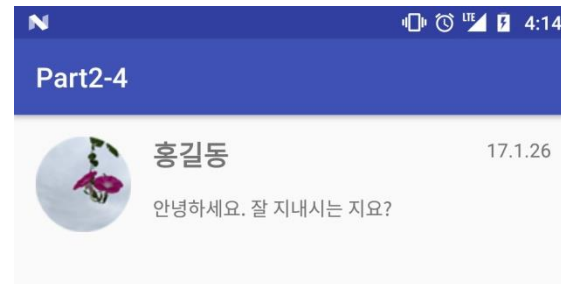
    <Button
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_weight="50"
        android:text="View3" />

</LinearLayout>
```



목록 화면을 LinearLayout을 적용해 작성

1. Module 생성
2. 이미지 리소스 파일 복사
3. activity\_main.xml 파일 작성
4. 실행







## 2. 리니어레이아웃 ▶ Java 코드로 화면 만들기



### 실습 5-1 XML 없이 화면 코딩하기

- ❖ 버튼을 클릭하면 토스트 메시지가 출력되는 화면을 Java로 코딩해보자.
- ❖ 안드로이드 프로젝트 생성
  - ✓ 프로젝트 이름 : Project5\_1
  - ✓ 패키지 이름 : com.cookandroid.project5\_1
- ❖ 화면 디자인 및 편집
  - ✓ main.xml 삭제
- ❖ Java 코드 작성 및 수정
  - ✓ main.xml 삭제했기 때문에 오류가 발생함
  - ✓ //를 붙여 주석으로 처리한 후 진행

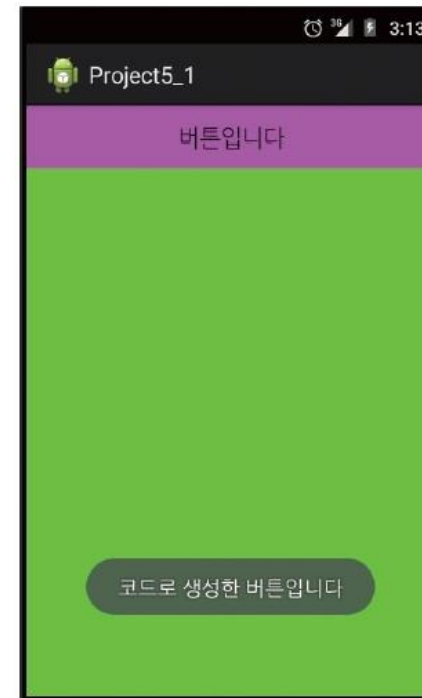


그림 5-7 Java 코드만 사용한 프로젝트

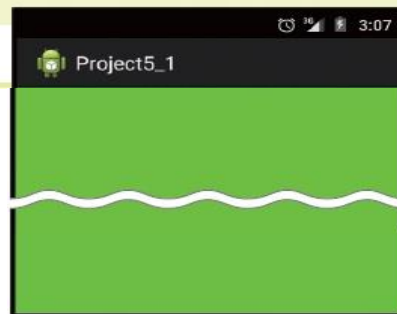
## 2. 리니어레이아웃 ▶ Java 코드로 화면 만들기



### 실습 5-1 XML 없이 화면 코딩하기

예제 5-8 리니어레이아웃을 생성하는 Java 코드

```
1 public void onCreate(Bundle savedInstanceState) {  
2     super.onCreate(savedInstanceState);  
3     // setContentView(R.layout.activity_main);  
4  
5     LinearLayout.LayoutParams params = new LinearLayout.LayoutParams(  
6         LinearLayout.LayoutParams.MATCH_PARENT,  
7         LinearLayout.LayoutParams.MATCH_PARENT);  
8  
9     LinearLayout baseLayout = new LinearLayout(this);  
10    baseLayout.setOrientation(LinearLayout.VERTICAL);  
11    baseLayout.setBackgroundColor(Color.rgb(0, 255, 0));  
12    setContentView(baseLayout,params);  
13 }
```



## 2. 리니어레이아웃 ▶ Java 코드로 화면 만들기



### 실습 5-1 XML 없이 화면 코딩하기

- ❖ 버튼을 만들고, 버튼을 클릭했을 때 토스트 메시지를 작성 (onCreate( ) 안에 이어서 코딩)

#### 예제 5-9 버튼을 생성하는 Java 코드

```
1 Button btn = new Button(this);
2 btn.setText("버튼입니다");
3 btn.setBackgroundColor(Color.MAGENTA);
4 baseLayout.addView(btn);
5
6 btn.setOnClickListener(new View.OnClickListener() {
7     public void onClick(View arg0) {
8         Toast.makeText(getApplicationContext(),
9             "코드로 생성한 버튼입니다", Toast.LENGTH_SHORT).show();
10    }
11 });
```



## 2. 리니어레이아웃 ▶ Java 코드로 하면 만들기

이름을 입력하세요

LinearLayout 수평 배치

이름을 입력하세요

res/layout/activity\_main.xml

```
<LinearLayout xmlns:android="http://..."
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="20px">
```

```
<LinearLayout android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```

```
<TextView android:text="이름을 입력하세요"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"/>
```

```
<EditText android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"/>
```

```
</LinearLayout>
```

```
<Button android:text="저장하기"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

```
</LinearLayout>
```

src/MainActivity.java

...

```
LinearLayout nameInputLinear = new LinearLayout( this );
nameInputLinear.setOrientation( LinearLayout.HORIZONTAL );
LinearLayout.LayoutParams nameInputLp =
    new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT );
```

```
TextView nameTv = new TextView( this );
nameTv.setText( "이름을 입력하세요" );
LinearLayout.LayoutParams nameTextLp = new LinearLayout.LayoutParams(
    0,
    LinearLayout.LayoutParams.WRAP_CONTENT );
nameTextLp.weight = 1;
```

```
EditText nameEt = new EditText( this );
LinearLayout.LayoutParams nameEditLp = new LinearLayout.LayoutParams(
    0,
    LinearLayout.LayoutParams.WRAP_CONTENT );
nameEditLp.weight = 1;
```

```
nameInputLinear.addView( nameTv, nameTextLp );
nameInputLinear.addView( nameEt, nameEditLp );

rootLinear.addView( nameInputLinear, nameInputLp );
```

## 2. 리니어레이아웃 ▶ Java 코드로 화면 만들기

자바 소스로 콘텐츠 영역을 채워보자.

res/layout/activity\_main.xml

```
<LinearLayout xmlns:android="http://..."
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="20px">

    <LinearLayout android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView android:text="이름을 입력하세요 "
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"/>

        <EditText android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"/>

    </LinearLayout>

    <Button android:text="저장하기"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

|                                     |                      |
|-------------------------------------|----------------------|
| 이름을 입력하세요                           | <input type="text"/> |
| <input type="button" value="저장하기"/> |                      |

src/MainActivity.java

```
...
Button saveBtn = new Button( this );
saveBtn.setText( "저장하기" );
LinearLayout.LayoutParams saveBtnLp =
    new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT );

rootLinear.addView( saveBtn, saveBtnLp );
```



## 2. 리니어레이아웃 ▶ Java 코드로 화면 만들기



### ▶ 직접 풀어보기 5-3

다음 화면을 XML 파일 없이 Java 코드만 이용하여 완성하라.

- 레이아웃에는 에디트텍스트 1개와 버튼 1개, 텍스트뷰 1개를 생성한다.
- 버튼을 클릭하면 에디트텍스트에 써진 문자열이 텍스트뷰에 나타나도록 한다.



그림 5-8 XML 파일 없이 프로젝트 생성



### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

#### ❖ 개념

- 렐러티브레이아웃(RelativeLayout)은 상대레이아웃이라고도 부르는데, 이름처럼 레이아웃 내부에 포함된 위젯들을 상대적인 위치로 배치

#### ❖ 렐러티브레이아웃의 상하좌우에 배치

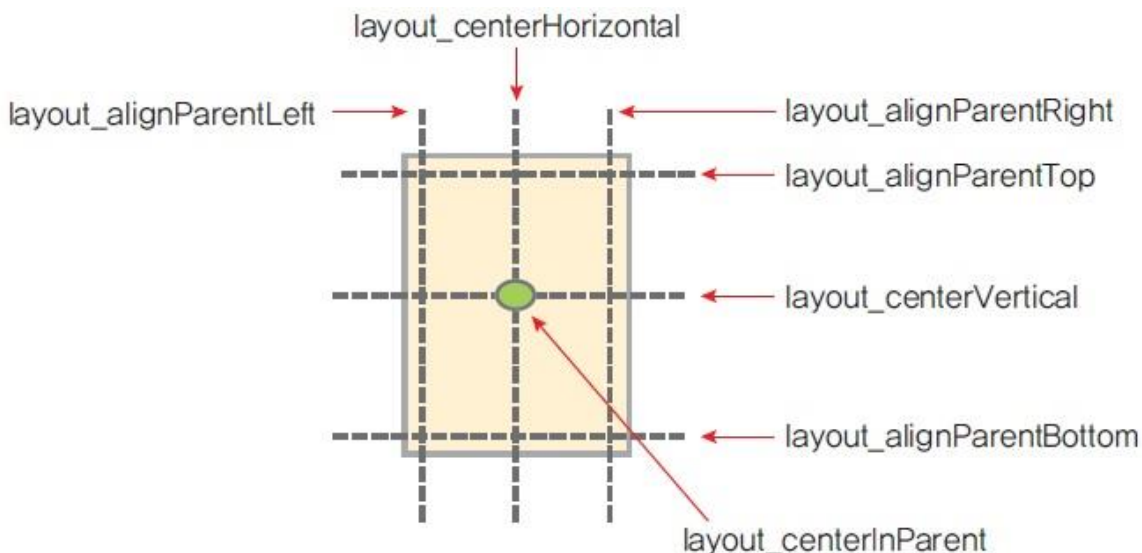


그림 5-9 부모(레이아웃)의 위치를 적용할 때 속성



### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

public Class  
**RelativeLayout**

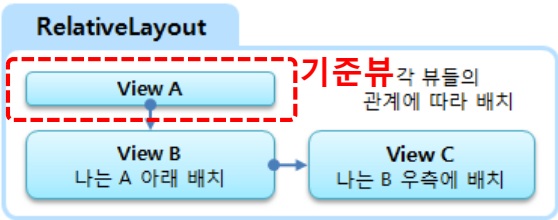
1. gravity  
2. ignoreGravity

public static Class  
**LayoutParams**

1. layout\_above  
2. layout\_alignBaseline  
3. layout\_alignBottom  
4. layout\_alignLeft  
5. layout\_alignParentBottom  
6. layout\_alignParentLeft  
7. layout\_alignParentRight  
8. layout\_alignParentTop  
9. layout\_alignRight  
10. layout\_alignTop  
11. layout\_below  
12. layout\_centerHorizontal  
13. layout\_centerInParent  
14. layout\_centerVertical  
15. layout\_leftOf  
16. layout\_rightOf  
17. layout\_alignWithParentIfMissing

| XML 속성                   | 의미               | 미리 보기 |
|--------------------------|------------------|-------|
| layout_alignParentLeft   | 부모 영역 내 좌측에 배치   |       |
| layout_centerHorizontal  | 부모 영역 내 수평 중앙 배치 |       |
| layout_alignParentRight  | 부모 영역 내 우측에 배치   |       |
| layout_alignParentTop    | 부모 영역 내 상단 배치    |       |
| layout_centerVertical    | 부모 영역 내 수직 중앙 배치 |       |
| layout_alignParentBottom | 부모 영역 내 하단 배치    |       |
| layout_centerInParent    | 부모 영역 내 정 중앙 배치  |       |

부모 뷰그룹과의 관계 배치  
속성들은 기준 자식뷰에  
대부분 설정된다.



|                                                                                |                                                                                 |                                                                                 |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <div>View</div> <div>layout_alignParentLeft<br/>layout_alignParentTop</div>    | <div>View</div> <div>layout_centerHorizontal<br/>layout_alignParentTop</div>    | <div>View</div> <div>layout_alignParentTop<br/>layout_alignParentRight</div>    |
| <div>View</div> <div>layout_centerVertical<br/>layout_alignParentLeft</div>    | <div>View</div> <div>layout_centerInParent</div>                                | <div>View</div> <div>layout_centerVertical<br/>layout_alignParentRight</div>    |
| <div>View</div> <div>layout_alignParentBottom<br/>layout_alignParentLeft</div> | <div>View</div> <div>layout_alignParentBottom<br/>layout_centerHorizontal</div> | <div>View</div> <div>layout_alignParentBottom<br/>layout_alignParentRight</div> |





### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

#### ❖ 예제 1

[예제 5-10] 렐러티브레이아웃 XML 코드 1

```
1 <RelativeLayout xmlns:android="http://~"
2   android:layout_width="match_parent"
3   android:layout_height="match_parent" >
4   <Button
5       android:layout_alignParentTop="true"
6       android:layout_centerHorizontal="true"
7       android:text="위쪽" />
8   <Button
9       android:layout_alignParentLeft="true"
10      android:layout_centerVertical="true"
11      android:text="좌측" />
12   <Button
13       android:layout_centerInParent="true"
14       android:text="중앙" />
15   <Button
16       android:layout_alignParentRight="true"
17       android:layout_centerVertical="true"
18       android:text="우측" />
19   <Button
20       android:layout_alignParentBottom="true"
21       android:layout_centerHorizontal="true"
22       android:text="아래" />
23 </RelativeLayout>
```





### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

#### ❖ 다른 위젯의 상대위치에 배치

- 각 속성의 값은 다른 위젯의 id를 지정하면 되는데, “@+id/기준위젯\_아이디” 형식으로 사용
- 아래 그림을 간략하게 표현하기 위해서 나타난 속성 이름에서 앞의 “layout\_”은 생략하였음

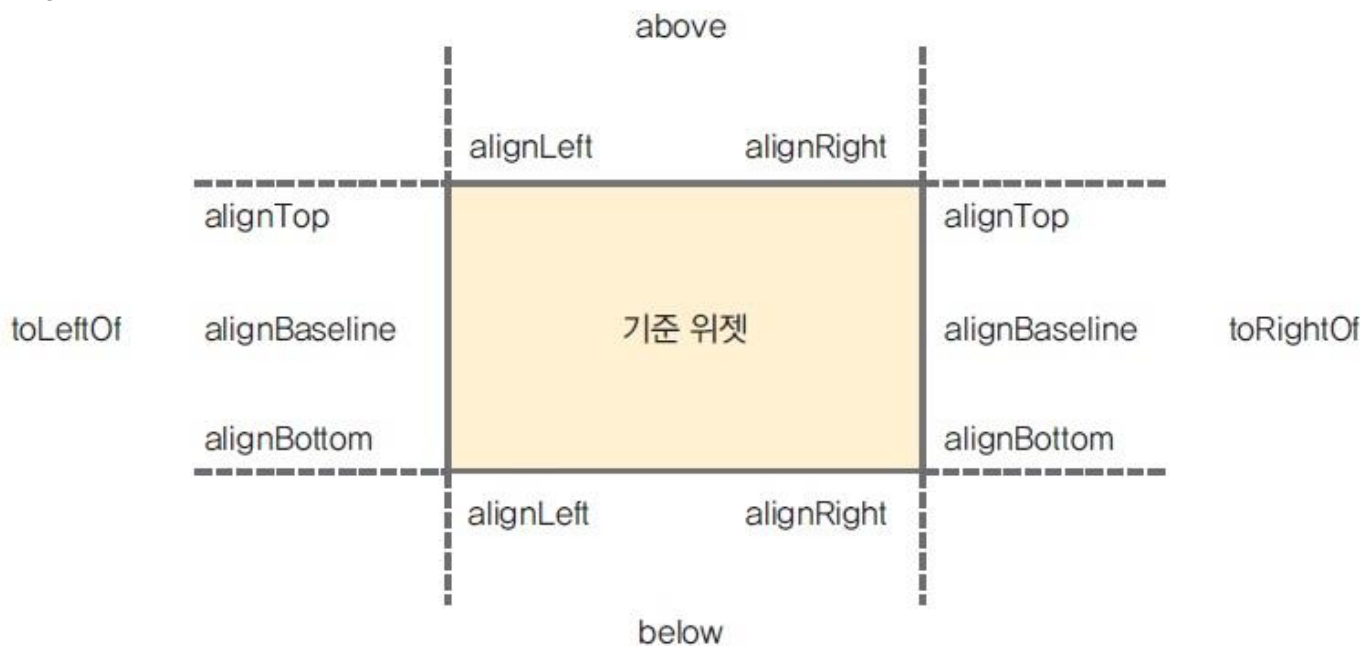


그림 5-10 다른 위젯에서 상대적인 위치를 적용할 때의 속성



# 상대 레이아웃에서 사용할 수 있는 속성들

[상대 레이아웃에서 부모 컨테이너와의 상대적 위치를 이용하는 속성]

| 속성                       | 설 명                            |
|--------------------------|--------------------------------|
| layout_alignParentTop    | - 부모 컨테이너의 위쪽과 뷰의 위쪽을 맞춤       |
| layout_alignParentBottom | - 부모 컨테이너의 아래쪽과 뷰의 아래쪽을 맞춤     |
| layout_alignParentLeft   | - 부모 컨테이너의 왼쪽 끝과 뷰의 왼쪽 끝을 맞춤   |
| layout_alignParentRight  | - 부모 컨테이너의 오른쪽 끝과 뷰의 오른쪽 끝을 맞춤 |
| layout_centerHorizontal  | - 부모 컨테이너의 수평 방향 중앙에 배치함       |
| layout_centerVertical    | - 부모 컨테이너의 수직 방향 중앙에 배치함       |
| layout_centerInParent    | - 부모 컨테이너의 수평과 수직 방향 중앙에 배치함   |



# 상대 레이아웃에서 사용할 수 있는 속성들

[상대 레이아웃에서 다른 뷰와의 상대적 위치를 이용하는 속성]

| 속성                   | 설 명                                 |
|----------------------|-------------------------------------|
| layout_above         | - 지정한 뷰의 위쪽에 배치함                    |
| layout_below         | - 지정한 뷰의 아래쪽에 배치함                   |
| layout_toLeftOf      | - 지정한 뷰의 왼쪽에 배치함                    |
| layout_toRightOf     | - 지정한 뷰의 오른쪽에 배치함                   |
| layout_alignTop      | - 지정한 뷰의 위쪽과 맞춤                     |
| layout_alignBottom   | - 지정한 뷰의 아래쪽과 맞춤                    |
| layout_alignLeft     | - 지정한 뷰의 왼쪽과 맞춤                     |
| layout_alignRight    | - 지정한 뷰의 오른쪽과 맞춤                    |
| layout_alignBaseline | - 지정한 뷰와 내용물의 아래쪽 기준선(baseline)을 맞춤 |



### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

#### ❖ 예제 2

[예제 5-11] 렐러티브레이아웃 XML 코드 2

```
1 <RelativeLayout xmlns:android="http://~"
2   android:layout_width="match_parent"
3   android:layout_height="match_parent" >
4   <Button
5     android:id="@+id/baseBtn"
6     android:layout_width="150dp"
7     android:layout_height="150dp"
8     android:layout_centerHorizontal="true"
9     android:layout_centerVertical="true"
10    android:text="기준 위젯" />
11   <Button
12     android:layout_alignTop="@+id/baseBtn"
13     android:layout_toLeftOf="@+id/baseBtn"
14     android:text="1번" />
15   ~~~~ 중간 생략 (버튼 2개) ~~~~
16   <Button
17     android:layout_above="@+id/baseBtn"
18     android:layout_alignLeft="@+id/baseBtn"
19     android:text="4번" />
20   <Button
21     android:layout_alignRight="@+id/baseBtn"
22     android:layout_below="@+id/baseBtn"
23     android:text="5번" />
24   <Button
25     android:layout_above="@+id/baseBtn"
26     android:layout_toRightOf="@+id/baseBtn"
27     android:text="6번" />
28 </RelativeLayout>
```



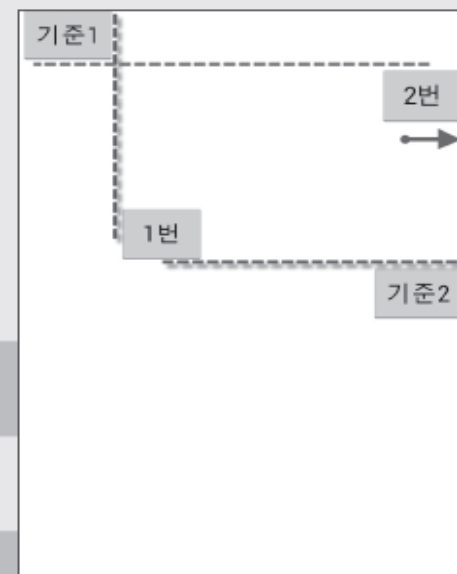


### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

#### ❖ 예제 3

[예제 5-12] 렐러티브레이아웃 속성을 조합한 XML 코드

```
1 <RelativeLayout xmlns:android="http://~"
2   android:layout_width="match_parent"
3   android:layout_height="match_parent" >
4   <Button
5     android:id="@+id/baseBtn1"
6     android:layout_alignParentLeft="true"
7     android:layout_alignParentTop="true"
8     android:text="기준1" />
9   <Button
10    android:id="@+id/baseBtn2"
11    android:layout_alignParentRight="true"
12    android:layout_centerVertical="true"
13    android:text="기준2" />
14   <Button
15     android:layout_above="@+id/baseBtn2"
16     android:layout_toRightOf="@+id/baseBtn1"
17     android:text="1번" />
18   <Button
19     android:layout_alignParentRight="true"
20     android:layout_below="@+id/baseBtn1"
21     android:text="2번" />
22 </RelativeLayout>
```







### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

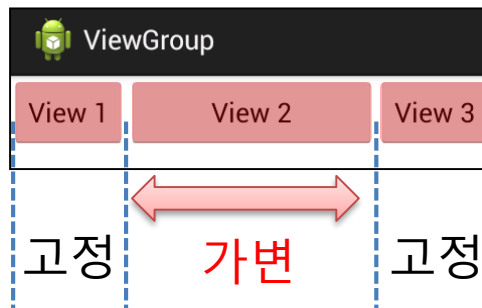
❶ 레이아웃의 유연성 속성은 기억해주세요.

res/layout/RelativeLayout\_flexible\_layout.xml

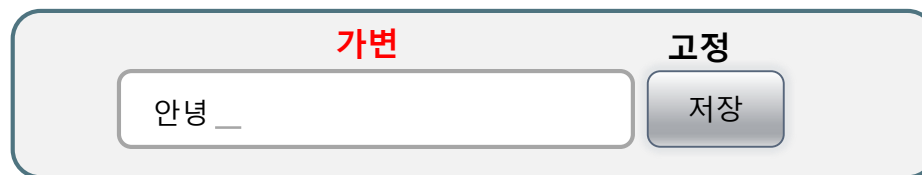
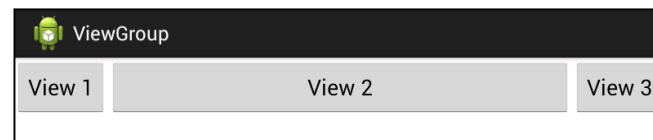
```
<RelativeLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <Button android:id="@+id/view1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:text="View 1"/>

    <Button android:id="@+id/view3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"
        android:text="View 3" />

    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_toRightOf="@id/view1"
        android:layout_toLeftOf="@id/view3"
        android:text="View 2" />
</RelativeLayout>
```



단말기를 가로로 회전했을 때



LinearLayout LayoutParams에 layout\_weight 속성이 있다면  
RelativeLayout LayoutParams은 각종 조합으로 레이아웃 유연성을 지원한다.

매우 빈번히 사용되니 꼭 기억해두자.





### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

## RelativeLayout LayoutParams 속성 – layout\_alignWithParentIfMissing 속성

res/layout/relativelayout\_relation\_view\_gone.xml

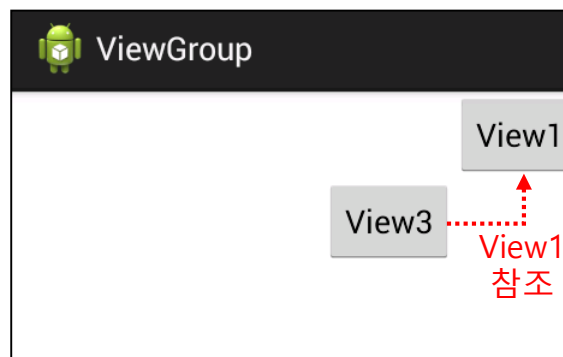
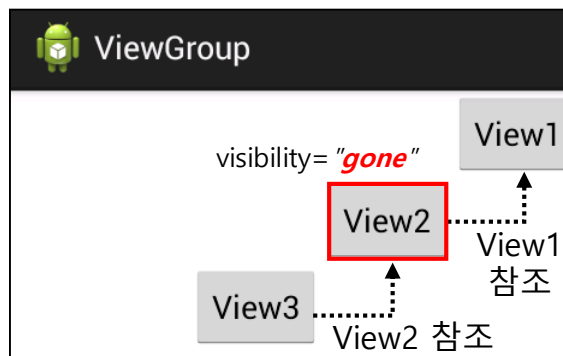
```
<RelativeLayout xmlns:android="..."
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <Button android:id="@+id/view1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"
        android:layout_alignParentTop="true"
        android:text="View1"/>

    <Button android:id="@+id/view2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/view1"
        android:layout_below="@id/view1"
        android:visibility="visible"
        android:text="View2"/>

    <Button android:id="@+id/view3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/view2"
        android:layout_below="@id/view2"
        android:text="View3"/>

</RelativeLayout>
```





### 3. 기타 레이아웃 ▶ 렐러티브레이아웃

res/layout/RelativeLayout\_layout\_align\_with\_parent\_if\_missing.xml

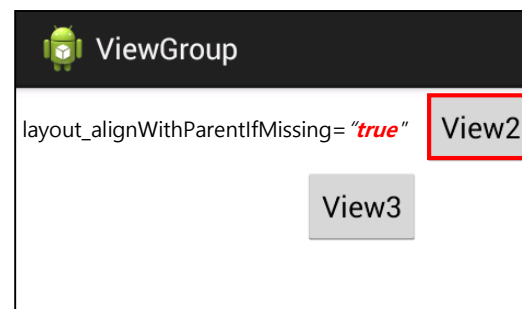
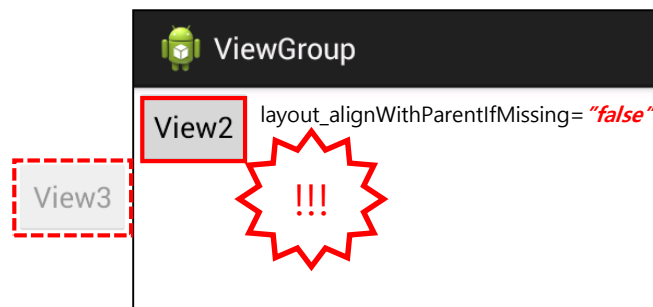
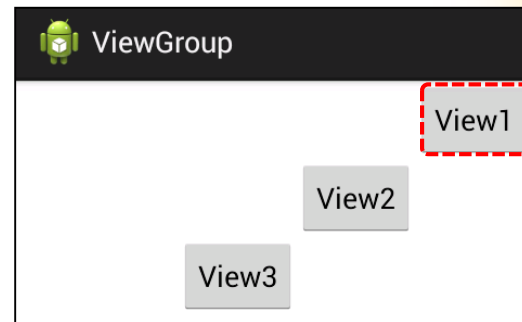
```
<RelativeLayout xmlns:android="..."
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <Button android:id="@+id/view1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentRight="true"
        android:layout_alignParentTop="true"
        android:visibility="gone"
        android:text="View1"/>

    <Button android:id="@+id/view2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/view1"
        android:layout_below="@id/view1"
        android:layout_alignWithParentIfMissing="false"
        android:text="View2"/>

    <Button android:id="@+id/view3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@id/view2"
        android:layout_below="@id/view2"
        android:text="View3"/>

</RelativeLayout>
```



참조했던 뷰의 속성을 따라간다.

사실 layout\_alignWithParentIfMissing 속성을 자주 사용하지는 않는다.

해당 속성을 활용한 기억이 손에 꼽을 정도다.

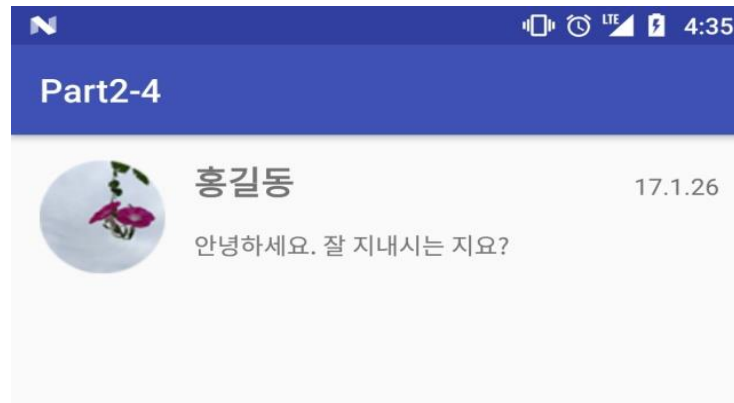
하지만 RelativeLayout은 관계형 배치 방식이므로 그 관계가 끊어진 경우를 반드시 고려해야겠다.



# Self-Check

RelativeLayout을 이용한 UI 구성

1. Activity 생성
2. activity\_lab4\_2.xml 작성
3. Lab4\_2Activity.java 실행

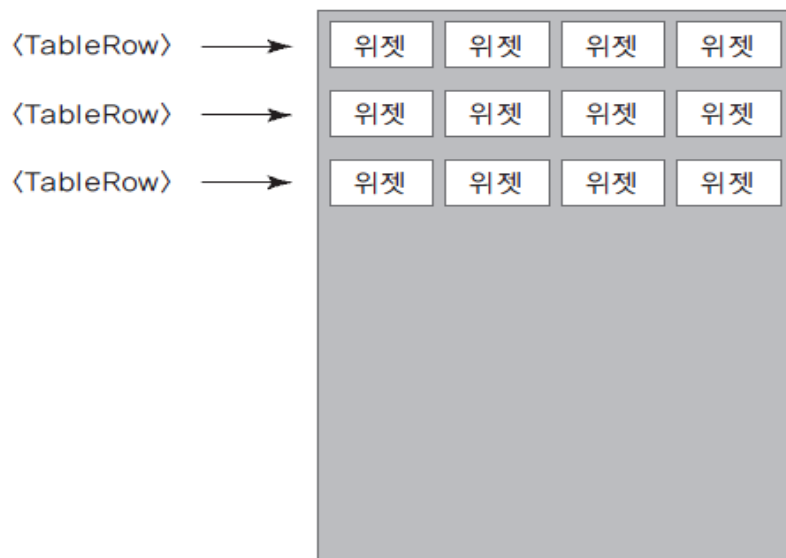




### 3. 기타 레이아웃 ▶ 테이블레이아웃

#### ❖ 개요

- 표 형태로 위젯을 배치할 때 주로 사용
- <TableRow>와 함께 사용되는데, <TableRow>의 개수가 바로 행의 개수가 된다. 열의 개수는 <TableRow>안에 포함된 위젯의 개수가 열의 개수로 결정된다.
- 3행 4열의 테이블레이아웃은 다음과 같다.



[그림 5-12] 테이블레이아웃 개요



### 3. 기타 레이아웃 ▶ 테이블레이아웃

#### ❖ 속성

- layout\_span은 열을 합쳐서 표시하라는 의미
- layout\_column은 지정된 열에 현재 위젯을 표시하라는 의미

[예제 5-13] 테이블레이아웃 XML 코드

```
1 <TableLayout xmlns:android="http://www."
2 >
3 <TableRow>
4 <Button
5     android:text="1" />
6 <Button
7     android:layout_span="2"
8     android:text="2" />
9 <Button
10    android:text="3" />
11 </TableRow>
```

```
12 <TableRow>
13 <Button
14     android:layout_column="1"
15     android:text="4" />
16 <Button
17     android:text="5" />
18 <Button
19     android:text="6" />
20 </TableRow>
21 </TableLayout>
```

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
|   | 4 | 5 |
|   |   | 6 |



### 3. 기타 레이아웃 ▶ 테이블레이아웃

## TableLayout 기본 속성 - shrinkColumns 속성

res/layout/tablelayout\_shrink\_columns.xml

```
<TableLayout xmlns:android="..."
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:background="#FF0"
    android:shrinkColumns="1">
    <TableRow>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X0"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X1"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X2"/>
    </TableRow>
    <TableRow>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X0 abc"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X1 abc"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X2 abc"/>
    </TableRow>
</TableLayout>
```



ViewPager

|             |             |             |
|-------------|-------------|-------------|
| View0X0     | View0X1     | View0X2     |
| View1X0 abc | View1X1 abc | View1X2 abc |
|             |             |             |



ViewPager

|             |             |             |
|-------------|-------------|-------------|
| View0X0     | View0X1     | View0X2     |
| View1X0 abc | View1X1 abc | View1X2 abc |
|             |             |             |

android:shrinkColumns="1"



### 3. 기타 레이아웃 ▶ 테이블레이아웃

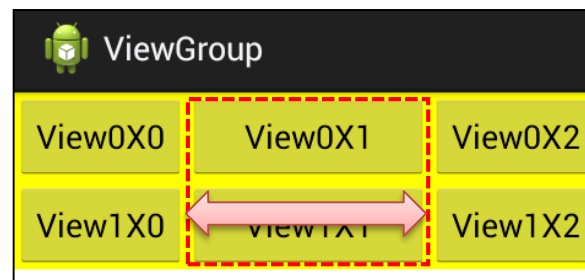
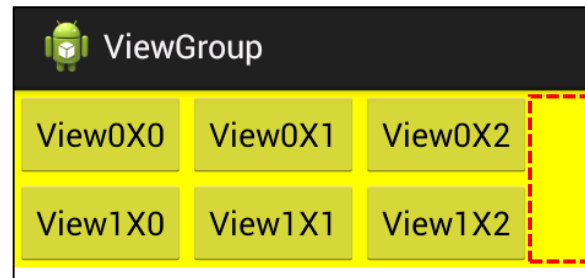
## TableLayout 기본 속성 - stretchColumns 속성

res/layout/tablelayout\_stretch\_columns.xml

```
<TableLayout xmlns:android="..."
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#FF0"
    android:stretchColumns="1">

    <TableRow>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X0"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X1"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View0X2"/>
    </TableRow>

    <TableRow>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X0"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X1"/>
        <Button android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="View1X2"/>
    </TableRow>
</TableLayout>
```



`android:stretchColumns="1"`

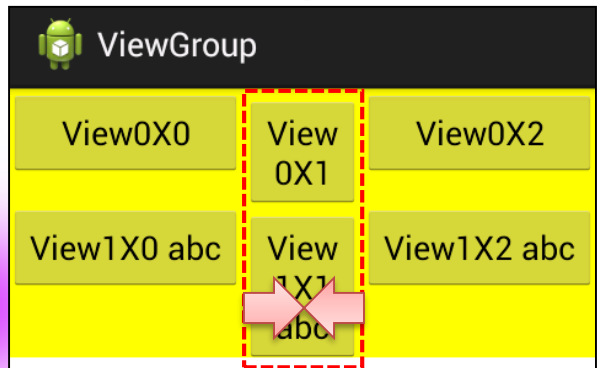
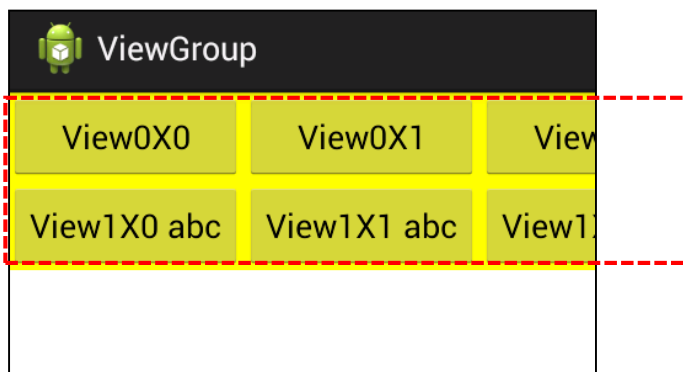




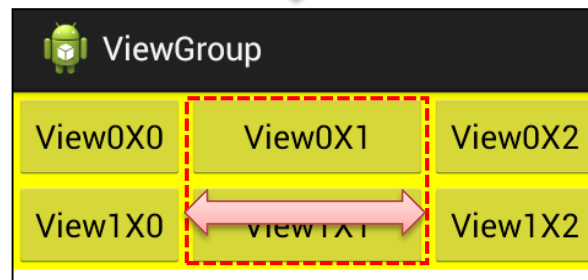
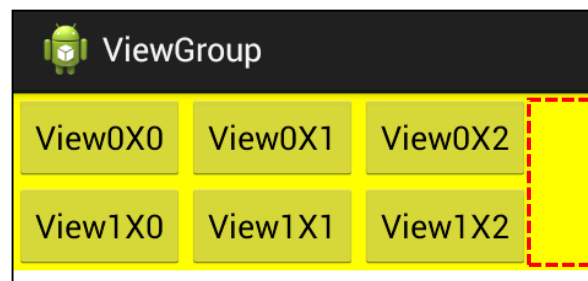
### 3. 기타 레이아웃 ▶ 테이블레이아웃

❶ 레이아웃의 유연성 속성은 기억해주세요.

TableLayout의  
shrinkColumns 속성



TableLayout의  
stretchColumns 속성





### 3. 기타 레이아웃 ▶ 테이블레이아웃



#### 실습 5-2 테이블레이아웃을 활용한 계산기 앱 만들기

❖ 테이블레이아웃을 활용하여 숫자 버튼까지 있는 계산기를 만들어보자.

❖ 안드로이드 프로젝트 생성

- ✓ 프로젝트 이름 : Project5\_2
- ✓ 패키지 이름 : com.cookandroid.project5\_2

❖ 화면 디자인 및 편집

- ✓ TableLayout 1개와 TableRow 9개로 구성
- ✓ 에디트텍스트 2개, 숫자 버튼 10개, 연산 버튼 4개, 텍스트뷰 1개를 생성
- ✓ 연산 버튼 위젯에는 layout\_margin을 적절히 지정
- ✓ 결과를 보여줄 TextView는 색상을 빨간색, 글자 크기는 20dp
- ✓ 각 위젯의 id는 위에서부터 Edit1, Edit2, BtnNum0~9, BtnAdd, BtnSub, BtnMul, BtnDiv, TextResult



그림 5-13 테이블레이아웃 계산기 결과 화면



### 3. 기타 레이아웃 ▶ 테이블레이아웃



#### 실습 5-2 테이블레이아웃을 활용한 계산기 앱 만들기

#### ❖ 화면 디자인 및 편집

예제 5-14 activity\_main.xml

```

1 <TableLayout xmlns:android="http://www."
2   android:layout_width="match_parent"
3   android:layout_height="match_parent" >
4
5   <TableRow>
6       <EditText
7           android:id="@+id/Edit1"
8           android:layout_span="5"
9           android:hint="숫자1 입력" />
10  </TableRow>
11  ~~~~ 중간 생략 (TableRow 1개, EditText 1개) ~~~~
12
13  <TableRow>
14      <Button
15          android:id="@+id/BtnNum0"
16          android:text="0" />
17      ~~~~ 중간 생략 (숫자 Button 4개) ~~~~
18  </TableRow>
19
20  <TableRow>
21      ~~~~ 중간 생략 (숫자 Button 5개) ~~~~
22

```

|        |   |   |   |   |
|--------|---|---|---|---|
| 숫자1 입력 |   |   |   |   |
| 숫자2 입력 |   |   |   |   |
| 0      | 1 | 2 | 3 | 4 |
| 5      | 6 | 7 | 8 | 9 |
| 더하기    |   |   |   |   |
| 빼기     |   |   |   |   |
| 곱하기    |   |   |   |   |
| 나누기    |   |   |   |   |
| 계산 결과: |   |   |   |   |

```

23 </TableRow>
24
25 <TableRow>
26     <Button
27         android:id="@+id/BtnAdd"
28         android:layout_margin="5dp"
29         android:layout_span="5"
30         android:text="더하기" />
31 </TableRow>
32
33 ~~~~ 중간 생략 (TableRow 3개, 연산 Button 3개) ~~~~
34
35 <TableRow>
36     <TextView
37         android:id="@+id/TextResult"
38         android:layout_margin="5dp"
39         android:layout_span="5"
40         android:text="계산 결과 : "
41         android:textColor="#FF0000"
42         android:textSize="20dp" />
43 </TableRow>
44 </TableLayout>

```



## 3. 기타 레이아웃 ▶ 테이블레이아웃



### 실습 5-2 테이블레이아웃을 활용한 계산기 앱 만들기

#### ❖ Java 코드 작성 및 수정

##### ✓ 전역변수 선언

- 숫자 버튼을 제외한 activity\_main.xml의 7개 위젯에 대응할 위젯변수 7개
- 입력될 2개 문자열 저장할 문자열 변수 2개
- 계산 결과를 저장할 정수 변수 1개
- 10개 숫자 버튼을 저장할 버튼 배열
- 10개 버튼의 id를 저장할 정수형 배열
- 증가 값으로 사용할 정수 변수

예제 5-15 Java 코드 1

```
1  ~~~~ 중간 생략 ~~~~
2  public class MainActivity extends Activity {
3      EditText edit1, edit2;
4      Button btnAdd, btnSub, btnMul, btnDiv;
5      TextView textResult;
6      String num1, num2;
7      Integer result;
8      Button[] numButtons = new Button[10];
9      Integer[] numBtnIDs = { R.id.BtnNum0, R.id.BtnNum1, R.id.BtnNum2, R.id.BtnNum3,
10         R.id.BtnNum4, R.id.BtnNum5, R.id.BtnNum6, R.id.BtnNum7,
11         R.id.BtnNum8, R.id.BtnNum9};
12     int i;
13
14
15     @Override
16     public void onCreate(Bundle savedInstanceState) {
17         ~~~~ 중간 생략 ~~~~
```



### 3. 기타 레이아웃 ▶ 테이블레이아웃



#### 실습 5-2 테이블레이아웃을 활용한 계산기 앱 만들기

##### ❖ Java 코드 작성 및 수정

✓ 숫자 버튼이 없다고 가정하고 연산 버튼을 터치했을 때의 내용을 코딩

예제 5-16 Java 코드 2

```
1  ~~~~ 중간 생략 ~~~~
2  public void onCreate(Bundle savedInstanceState) {
3      super.onCreate(savedInstanceState);
4      setContentView(R.layout.activity_main);
5
6      setTitle("테이블레이아웃 계산기");
7
8      edit1 = (EditText) findViewById(R.id.Edit1);
9      edit2 = (EditText) findViewById(R.id.Edit2);
10     btnAdd = (Button) findViewById(R.id.BtnAdd);
11     ~~~~ 중간 생략 (연산 버튼 3개 대입) ~~~~
12     textResult = (TextView) findViewById(R.id.TextResult);
13
14     btnAdd.setOnTouchListener(new View.OnTouchListener() {
15         public boolean onTouch(View arg0, MotionEvent arg1) {
16             num1 = edit1.getText().toString();
17             num2 = edit2.getText().toString();
18             result = Integer.parseInt(num1) + Integer.parseInt(num2);
19             textResult.setText("계산 결과 : " + result.toString());
20             return false;
21         }
22     });
23     ~~~~ 중간 생략 (연산 버튼 3개 터치 이벤트 리스너) ~~~~
24 }
```





### 3. 기타 레이아웃 ▶ 테이블레이아웃



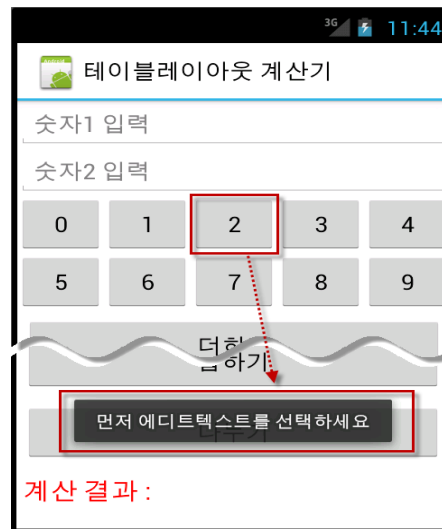
#### 실습 5-2 테이블레이아웃을 활용한 계산기 앱 만들기

##### ❖ Java 코드 작성 및 수정

✓ 숫자 버튼 10개를 배열 변수에 대입한 후에 각 버튼의 클릭 이벤트 리스너를 만들

예제 5-17 Java 코드 3

```
1 for (i = 0; i < numBtnIDs.length; i++) {  
2     numButtons[i] = (Button) findViewById(numBtnIDs[i]);  
3 }  
4  
5 for (i = 0; i < numBtnIDs.length; i++) {  
6     final int index; // 주의! 꼭 필요함..  
7     index = i;  
8  
9     numButtons[index].setOnClickListener(new View.OnClickListener() {  
10        public void onClick(View v) {  
11  
12            if (edit1.isFocused() == true) {  
13                num1 = edit1.getText().toString()  
14                    + numButtons[index].getText().toString();  
15                edit1.setText(num1);  
16            } else if (edit2.isFocused() == true) {  
17                num2 = edit2.getText().toString()  
18                    + numButtons[index].getText().toString();  
19                edit2.setText(num2);  
20            } else {  
21                Toast.makeText(getApplicationContext(),  
22                    "먼저 에디트텍스트를 선택하세요", Toast.LENGTH_SHORT).show();  
23            }  
24        }  
25    });  
26  
27 }
```

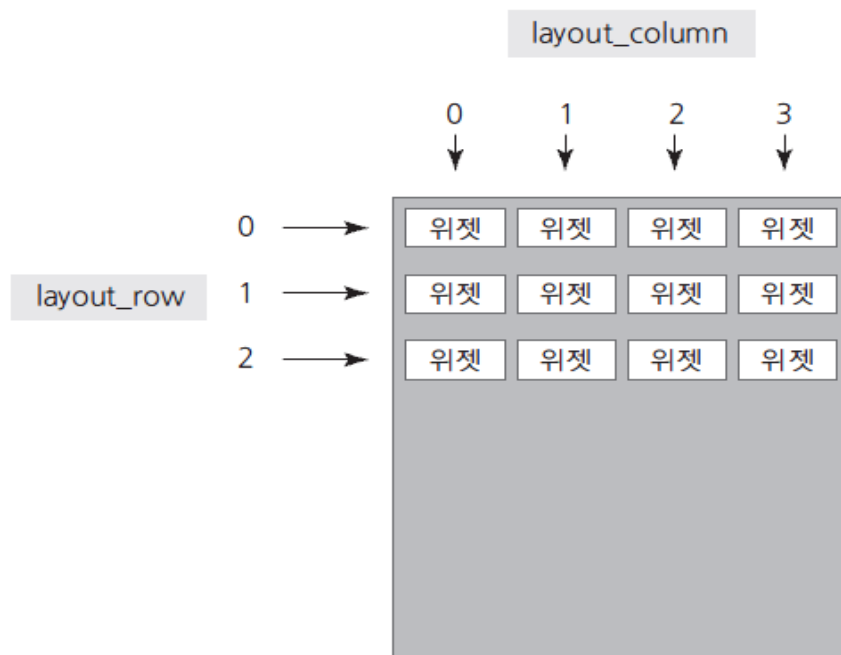




### 3. 기타 레이아웃 ▶ 그리드레이아웃

#### ❖ 개요

- 테이블레이아웃에서는 좀 어려웠던 행을 확장하는 기능도 간단하게 할 수 있어서 유연한 화면 구성에 적합
- 행, 열을 지정하는 것은 상당히 직관적
- Android 4.0 (아이스크림 샌드위치, API 14)부터 지원.



[그림 5-15] 그리드레이아웃 개요





### 3. 기타 레이아웃 ▶ 그리드레이아웃

#### ❖ 그리드레이아웃 속성

- ✓ <GridLayout> 자체에 자주 사용되는 속성
  - rowCount : 행 개수
  - columnCount : 열 개수
  - orientation : 그리드를 수평 방향을 우선할지, 수직 방향을 우선할지를 결정
  
- ✓ 그리드레이아웃 안에 포함될 위젯에서 자주 사용되는 속성
  - layout\_row : 자신이 위치할 행 번호(0번부터 시작)
  - layout\_column : 자신이 위치할 열 번호(0번부터 시작)
  - layout\_rowSpan : 행을 지정된 개수만큼 확장
  - layout\_columnSpan : 열을 지정된 개수만큼 확장
  - layout\_gravity : 주로 fill, fill\_vertical, fill\_horizontal 등으로 지정  
행 또는 열 확장시, 위젯을 확장된 셀에 꽉 채우는 효과를 냄

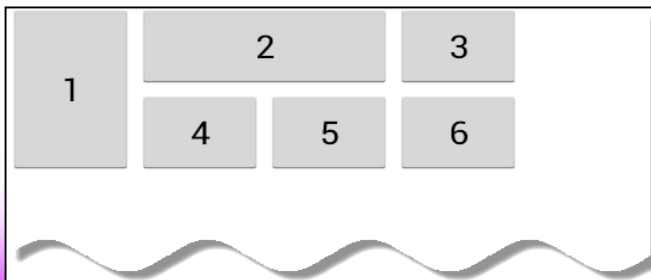


### 3. 기타 레이아웃 ▶ 그리드레이아웃

#### ❖ 예제

[예제 5-18] 그리드레이아웃 XML 코드

```
1 <GridLayout xmlns:android="http://www."
2     android:columnCount="4"
3     android:rowCount="2" >
4     <Button
5         android:layout_column="0"
6         android:layout_row="0"
7         android:layout_rowSpan="2"
8         android:layout_gravity="fill_vertical"
9         android:text="1" />
```



```
10 <Button
11     android:layout_column="1"
12     android:layout_row="0"
13     android:layout_columnSpan="2"
14     android:layout_gravity="fill_horizontal"
15     android:text="2" />
16 <Button
17     android:layout_column="3"
18     android:layout_row="0"
19     android:text="3" />
20 <Button
21     android:layout_column="1"
22     android:layout_row="1"
23     android:text="4" />
24     ~~~~ 중간 생략 ~~~~
25 </GridLayout>
```



### 3. 기타 레이아웃 ▶ 그리드레이아웃



#### ▶ 직접 풀어보기 5-5

[실습 5-2]를 그리드레이아웃으로 변경해서 실행해보자.

**HINT** Java 코드는 변경할 필요가 없고, XML만 변경하면 된다. XML 위젯의 id도 동일하게 사용한다.

22

37

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 | 9 |

더하기

빼기

곱하기

나누기

계산 결과 : 59

그림 5-16 그리드레이아웃 계산기



### 3. 기타 레이아웃 ▶ 프레임레이아웃

#### ❖ 프레임레이아웃(FrameLayout)

- ✓ 단순히 레이아웃 내의 위젯들은 왼쪽 상단부터 겹쳐서 출력
- ✓ 프레임레이아웃 자체로 사용하기보다는 탭 위젯 등과 혼용해서 사용할 때 유용



그림 5-17 프레임레이아웃 개요



# 프레임 레이아웃과 뷰의 전환

## 뷰 전환 예제

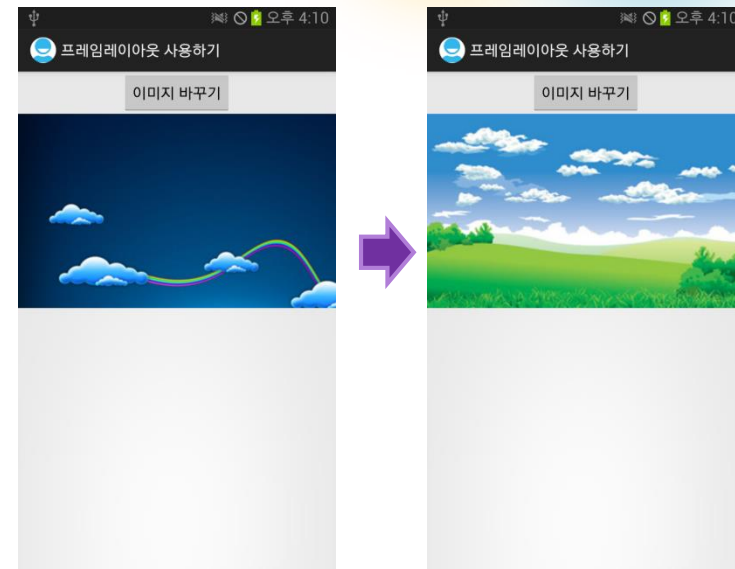
- 프레임 레이아웃을 이용해 뷰를 중첩하여 만들기
- 버튼을 누르면 다른 이미지로 전환하기

### XML 레이아웃

- 레이아웃 코드 작성

### 메인 액티비티 코드

- 메인 액티비티 코드 작성



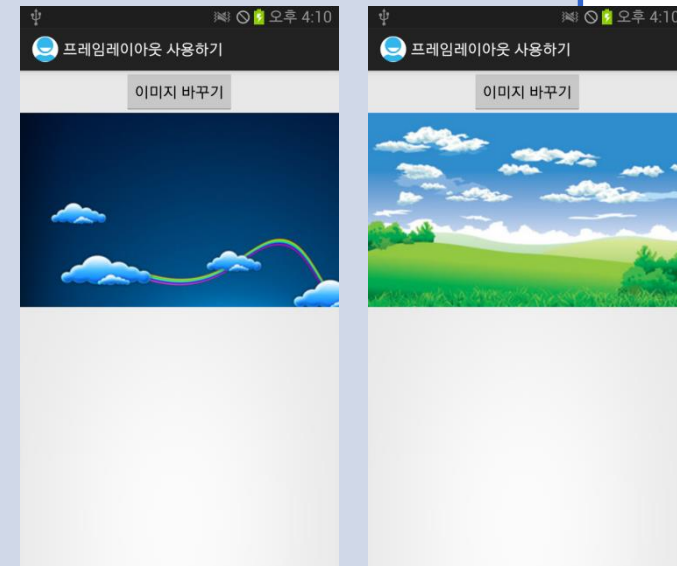


# 프레임 레이아웃과 뷰의 전환 – XML 레이아웃

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    >
    <Button
        android:id="@+id/button01"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Change Image"
    />
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
    >
```

1 전환 버튼

2 화면 채우기



Continued..



# 프레임 레이아웃과 뷰의 전환 – XML 레이아웃

```
<ImageView  
    android:id="@+id/imageView01"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:src="@drawable/dream01"  
    android:visibility="invisible"
```

```
/>
```

```
<ImageView  
    android:id="@+id/imageView02"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:src="@drawable/dream02"  
    android:visibility="visible"
```

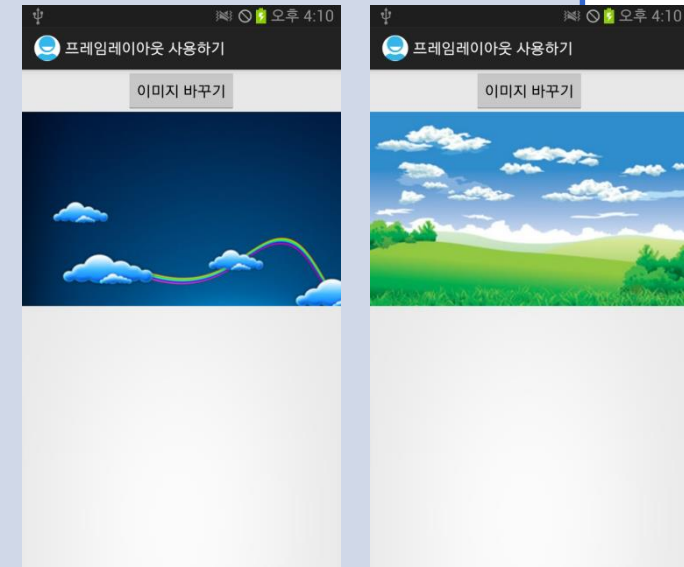
```
/>
```

```
</FrameLayout>
```

```
</LinearLayout>
```

3 이미지 뷰 설정

4 이미지 뷰 설정







# 프레임 레이아웃과 뷰의 전환 – 메인 액티비티 코드

```
package org.androidtown.ui;

...

public class SampleFrameLayoutActivity extends Activity {
    Button button01;
    ImageView imageView01;
    ImageView imageView02;
    int imageIndex = 0;

    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        button01 = (Button) findViewById(R.id.button01);
        imageView01 = (ImageView) findViewById(R.id.imageView01);
        imageView02 = (ImageView) findViewById(R.id.imageView02);
        button01.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
                changeImage();
            }
        });
    }
}
```



1

객체 참조

Continued..



# 프레임 레이아웃과 뷰의 전환 – 메인 액티비티 코드

```
private void changeImage() {
```

```
    if (imageIndex == 0) {
```

```
        imageView01.setVisibility(View.VISIBLE);
```

```
        imageView02.setVisibility(View.INVISIBLE);
```

```
        imageIndex = 1;
```

```
    } else if (imageIndex == 1) {
```

```
        imageView01.setVisibility(View.INVISIBLE);
```

```
        imageView02.setVisibility(View.VISIBLE);
```

```
        imageIndex = 0;
```

```
    }
```

```
}
```

```
}
```



2 이미지 뷰 설정



3 이미지 뷰 설정



## 한 화면에 두개의 이미지 뷰 배치하기

- ▶ 두 개의 이미지뷰 사이에 버튼을 두 개 만들고 오른쪽 버튼을 누르면 상단의 이미지가 하단으로 옮겨져 보이고 왼쪽 버튼을 누르면 상단으로 다시 옮겨지는 기능을 추가하시오.

- ▷ 화면을 위와 아래 두 영역으로 나누고 그 영역에 각각 이미지뷰를 배치합니다.
- ▷ 각각의 이미지뷰는 스크롤이 생길 수 있도록 합니다.
- ▷ 상단의 이미지뷰에 하나의 이미지가 보이도록 합니다.
- ▷ 두개의 이미지뷰 사이에 버튼을 하나 만들고 그 버튼을 누르면 상단의 이미지가 하단으로 옮겨져 보이고 다시 누르면 상단으로 다시 옮겨지는 기능을 추가합니다.

